Check for updates

JAMES | Journal of Advances in Modeling Earth Systems®

RESEARCH ARTICLE
10.1029/2025MS005313

Key Points:

- We present a JAX-based differentiable multiphase Lattice Boltzmann library, scalable across CPUs, GPUs, and distributed systems
- A modified Shan-Chen pseudopotential incorporating arbitrary equations of state and tunable wettability and surface tension is employed
- The library features decoupled compute and write precision, and achieves giga-scale lattice updates per second on multi-GPU setups

Correspondence to:

P. Gentine and S. Kelly,
pg2328@columbia.edu;
sak2280@columbia.edu

Citation:

Pradhan, P., Gentine, P., & Kelly, S. (2026). JAX-LaB: A high-performance, differentiable Lattice Boltzmann library for modeling multiphase fluid dynamics in geosciences and engineering. *Journal of Advances in Modeling Earth Systems*, *18*, e2025MS005313. https://doi.org/10.1029/2025MS005313

Received 26 JUN 2025
Accepted 11 JAN 2026

© 2026 The Author(s). Journal of Advances in Modeling Earth Systems published by Wiley Periodicals LLC on behalf of American Geophysical Union. This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

JAX-LaB: A High-Performance, Differentiable Lattice Boltzmann Library for Modeling Multiphase Fluid Dynamics in Geosciences and Engineering

Piyush Pradhan[1], Pierre Gentine[1], and Shaina Kelly[1]

[1]Department of Earth and Environmental Engineering, Columbia University, New York, NY, USA

Abstract We introduce JAX-LaB, a differentiable, Python-based Lattice Boltzmann simulation library designed for modeling multiphase and multiphysics fluid dynamics problems in hydrologic, geologic, and engineered porous media settings. The library is designed as an extension to XLB (Ataei & Salehipour, 2024, https://doi.org/10.1016/j.cpc.2024.109187), and it is built on the JAX framework (Bradbury et al., 2018, http://github.com/jax-ml/jax). The library delivers a performant, hardware-agnostic implementation that seamlessly integrates with machine learning libraries and scales efficiently across CPUs, multi-GPU setups, and distributed environments. Multiphase interactions are modeled using the Shan-Chen pseudopotential method, coupled with an equation of state (EOS) to reproduce densities consistent with Maxwell's construction, enabling accurate simulation of flows with density ratios $>10^7$ while maintaining low spurious currents. Fluid wetting is achieved using the "improved" virtual density scheme (Q. Li et al., 2019, https://doi.org/10.1103/physreve.100.053313), which enables precise control of contact angle on flat and curved surfaces, while eliminating non-physical films seen in the Shan-Chen virtual density scheme. This scheme integrates directly into the interaction force calculations, removing the need to handle fluid-fluid and fluid-solid forces separately. We validate the library's accuracy and performance through comprehensive analytical benchmarks, including Laplace's law, capillary rise in parallel plates, and multi-component cocurrent flow in a channel. We then use the code for several applications involving multicomponent and multiphase flows, including permeability estimation, injection of supercritical CO_2 in a water-saturated Fontainebleau sandstone, and obtaining the characteristic curves for a sphere pack geometry. Finally, the single-GPU performance and multi-GPU scaling of the code are evaluated on both single-node and distributed systems. The library is open-source under the Apache license and available at https://github.com/piyush-ppradhan/JAX-LaB.

Plain Language Summary Several natural and engineering processes can be classified as multiphase flows involving either a single chemical entity (for instance, water) in multiple phases (typically liquid and gas) or distinct chemical entities such as water and air coexisting in distinct phases. For many such applications, numerical simulations are necessary, as experimental investigations can be challenging. To accurately model multiphase flows, it is essential not only to solve the governing equations that describe fluid dynamics, but also to properly capture phase distributions and account for complex fluid behaviors such as liquid-vapor interfaces, which continuously form, deform, and break during flow, and wettability, which governs fluid-solid interactions. In this work, we introduce a Python-based library to model multiphase systems. The library performs computation using JAX, a high-performance Python library for scientific computations. We describe the various numerical techniques implemented in the library and then benchmark them against several well-known problems with analytical solutions, highlighting the applicability of JAX-LaB for modeling many different multiphase fluid flows. We then showcase several exemplar problems where JAX-LaB can be used in the geosciences and beyond. Finally, we test the performance of the library on single and multiple Graphics Processing Units (GPUs).

1. Introduction

Natural and engineered fluid-flow processes frequently involve more than one phase or chemical species interacting simultaneously. For example, in nature, water migrates through soil as both liquid and vapor during evapotranspiration; permafrost ice thaws to release liquid water and gas (Song et al., 2016); microscopic droplets nucleate and grow into clouds (Yoshino et al., 2022); and large-scale ocean currents develop from coupled density and temperature gradients (Linhao et al., 2005). In engineering contexts, multiphase transport underpins enhanced

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

hydrocarbon recovery (Xu et al., 2020), phase-change desalination membranes separate fresh water from brine (H. Wang et al., 2024), and atomized fuel droplets evaporate and ignite within combustion chambers (Pitsch, 2006). Flows in which a single substance (e.g., water) coexists in multiple phases (for instance, liquid and vapor) are termed multiphase flows, whereas those involving distinct chemical components (e.g., CO_2 and water) in several phases are termed multicomponent multiphase flows. Both exhibit complex behaviors such as surface tension, capillarity, phase change (evaporation and condensation), and, in some cases, cavitation that govern mass, momentum, and energy transport across scales from pore spaces to the atmosphere or industrial reactors (Sukop & Thorne, 2006). Interfaces between these phases continually migrate, deform, break up, and merge, and the intricate geometries (e.g., porous rocks or soils) and extreme conditions (e.g., supercritical fluids) in many real-world systems often make experimental studies impractical. As a result, numerical modeling has become an essential tool for investigating multiphase flows (Aidun & Clausen, 2010), provided it accurately captures vapor-liquid fractions and their densities via an appropriate equation of state and reproduces interface dynamics and wetting behavior of fluids.

Python libraries such as NumPy (Harris et al., 2020), JAX (Bradbury et al., 2018), PyTorch (Paszke et al., 2019) and TensorFlow (Abadi et al., 2016) provide a high-level interface for performing computations on arrays (tensors) while leveraging highly optimized C/C++ libraries in the backend to target massively parallel hardware such as graphics processing unit (GPU) and tensor processing unit. This approach combines the performance of C/C++ with the ease of using Python and hence, finds extensive use in domains such as scientific computing and machine learning. Recent works from Kochkov et al. (2021) and Vinuesa and Brunton (2022) have highlighted the potential of machine learning to accelerate the performance of Computational Fluid Dynamics (CFD) codes and in predicting the spatio-temporal evolution of fluid physics, and this development is motivated by the emergence of powerful general-purpose automatic differentiation frameworks such as Tensorflow (Abadi et al., 2016), JAX (Bradbury et al., 2018), and PyTorch (Paszke et al., 2019). Yet, to date, numerical models for multicomponent multiphase flows have not been well integrated with ML techniques, which is primarily due to the use of different programming languages for machine Learning (Python) and CFD (C/C++/Fortran). Thus, there is a need to develop a single-language implementation of a high-performance, accurate, and differentiable CFD code that can integrate well with existing machine learning libraries to provide a unified workflow for forward and inverse modeling in CFD.

The Lattice Boltzmann method (LBM) is a widely used computational approach for simulating fluid flows across diverse scales and applications, including porous media, microfluidics, biological flows, and even large/exascale applications such as meteorological flows (Aidun & Clausen, 2010; Feng et al., 2019; Mittal & Iaccarino, 2005; Petersen & Brinkerhoff, 2021; Wörner, 2012; J. Zhang, 2011). LBM distinguishes itself from other well-known CFD methods, such as the finite volume method (FVM), by numerically solving a discretized form of the Boltzmann equation, known as the Lattice Boltzmann equation (LBE), instead of the Navier-Stokes equation. The main advantage of the LBM is its simple two-step algorithm: a local collision step followed by a streaming step that transfers data to neighboring nodes. This structure, along with its explicit time integration, makes LBM inherently parallel and highly efficient on modern accelerated hardware such as Graphics Processing Units (GPUs). Additionally, LBM utilizes uniform Cartesian grids instead of geometry-conforming meshes, making it an ideal choice for modeling complex or heterogeneous domains. Using the Shan-Chen pseudopotential method, LBM allows easy incorporation of numerous features to model real-world fluids, such as equation of state (Yuan & Schaefer, 2006), surface tension control (Hu et al., 2014; Q. Li & Luo, 2013) and interface dynamics such as formation, merging and deformation without performing computationally-expensive interface tracking (Coon et al., 2014). Moreover, LBM can be used to simulate turbulent flows, either directly (DNS) using specific collision model, such as the entropic KBC model (Karlin et al., 2014) or by incorporating turbulence models such as Reynolds-averaged Navier Stokes (Filippova et al., 2001; K. Li et al., 2012; Pellerin et al., 2015) and large eddy simulation (LES) (Bartlett et al., 2013; Premnath et al., 2009). In the context of hydrology, LBM has been effectively used to model evaporation from porous media (Fei et al., 2022; Tian et al., 2025), thawing of frozen soils (Song et al., 2016), sediment transport in soils (Zhou et al., 2021), cloud formation (Yoshino et al., 2022) and ocean circulation (Linhao et al., 2005). In the context of geohydrology or subsurface reservoir engineering, LBM has been effectively used to understand immiscible displacement, drainage, imbibition, and residual trapping during enhanced oil recovery from reservoirs (Liu et al., 2021), flow of supercritical carbon dioxide (CO_2) through rocks (An et al., 2021; Xie et al., 2017), identification of the representative elementary volume in low-permeability reservoirs such as shales (Kelly et al., 2016), and to validate continuum-scale models by deriving

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

effective properties like permeability, capillary pressure functions and relative permeability curves (Liu et al., 2021; T. Zhang et al., 2019). LBM can be coupled with other numerical solvers to model multiphysics systems. For example, LBM can be extended to handle the energy equation (thermal LBM) for simulating evaporation and boiling phenomena (Luo et al., 2021), or combined with the advection-diffusion equation to model reactive transport in porous media (Gao et al., 2017). Moreover, LBM is well-suited for array-based computation, making it highly compatible with machine learning frameworks for both forward and inverse modeling tasks. These include applications such as permeability estimation (Graczyk & Matyka, 2020), steady-state flow prediction (Ataei & Salehipour, 2024; Y. D. Wang et al., 2021), and capturing hysteresis behavior in porous materials (Chaaban et al., 2024). Machine learning can also enhance the performance of LBM by optimizing the relaxation rates according to the spatio-temporal characteristics of the flow field (Bedrunka et al., 2024; Horstmann et al., 2024). Finally, fast LBM solvers enable large-scale data generation, which can serve as training data sets for data-driven models.

Open-source libraries such as Palabos (Latt et al., 2021), OpenLB (Krause et al., 2021), MF-LBM(Chen et al., 2019), Taxila-LBM (Coon et al., 2014), Sailfish (Januszewski & Kostur, 2014), waLBerla (Bauer et al., 2021), MPLBM-UT (Santos, Gigliotti, et al., 2022) and ch4-project (Calzavarini, 2019) can run LBM simulation in parallel across CPUs and GPUs, but are written using C/C++ or Fortran and rely on vendor-specific libraries such as CUDA (Luebke, 2008). Platform-agnostic codes such as FluidX3D (Lehmann, 2019), which is based on the Open Compute Language (OpenCL) (Stone et al., 2010), also exist; however, it too is written in C++. In contrast, JAX-LaB, built as an extension of the single-phase differentiable LBM library XLB (Ataei & Salehipour, 2023), offers a Python-based solution that seamlessly targets various backends like CPU, GPU, multi-GPU, and distributed GPU systems without vendor-specific code modifications, enabling rapid prototyping and execution while allowing for easy solver customization. Furthermore, in contrast to other Python-based differentiable CFD libraries such as Lettuce (Bedrunka et al., 2021), Taichi-LBM3D (J. Yang et al., 2022), and JAX-Fluids (which is based on FVM) (Bezgin et al., 2023, 2025), JAX-LaB supports both single and multicomponent multiphase systems by incorporating any non-ideal equation of state and delivers superior performance on single-GPU platforms compared to Lettuce and Taichi-LBM3D.

2. Numerical Methods and Implementation

JAX-LaB models fluids using the LBM, which is paired with the Shan-Chen pseudopotential method to model multiphase-multicomponent systems. The pseudopotential computation is modified from the original formulation by Shan and Chen (1993) to incorporate non-ideal fluid behavior using an equation of state (EOS), and control the interfacial tension and fluid-solid wettabilty for multiple fluid phases. The code implements commonly used equations of state, including Carnahan-Starling, Peng-Robinson, Redlich-Kwong, Redlich-Kwong-Soave, and Van der Waals, enabling the modeling of high-density ratio flows $\left(>10^{8}\right)$ with minimal spurious currents $\left(<2\times10^{-3}\right)$ for a wide range of contact angles $(5-170°)$. In this section, the numerical methodology employed in JAX-LaB, including the available collision models and multiphase implementation, is described.

2.1. Lattice Boltzmann Method

LBM describes the evolution of the probability distribution function f by solving the LBE:

$$f_i(x+\mathbf{e_i}\Delta t, t+\Delta t)-f_i(x,t)=\Omega_i \quad i=0,1\ldots q-1 \tag{1}$$

Here, Ω is the collision operator. Distributions are evaluated at a fixed set of points on a Cartesian grid and for specific velocity directions $(\mathbf{e}_i)$, typically defined using the DdQq scheme, where d represents the dimension and q denotes the number of velocity directions considered. Some popular lattice velocity sets used in LBM include D2Q5 and D2Q9 in two dimensions and D3Q15, D3Q19 and D3Q27 in three dimensions. The LBE is numerically solved using a combination of *collision* (local) and *streaming* (non-local) steps. In the *collision* step, the distribution functions for the next timestep are determined using a collision model, while in the *streaming* step, these distributions are moved to their respective adjacent nodes along the directions defined in the discrete velocity set.

$$\begin{aligned}&\text{collision}: f_i^{\,*}(\mathbf{x},t)=f_i+\Omega_i\\&\text{streaming}: f_i(\mathbf{x}+\mathbf{e_i}\Delta t, t+\Delta t)=f_i^{\,*}(\mathbf{x},t)\end{aligned} \tag{2}$$

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

JAX-LaB supports various collision operators such as the single-relaxation time or Bhatnagar-Gross-Krook (BGK) operator, multi-relaxation time (MRT) operator (Coveney et al., 2002), and the central moment (cascaded) operator (CLBM) (Geier et al., 2006). The post-collision distribution using these collision models can be summarized as:

$$\begin{aligned}
&\text{BGK}: f_i{}^*(\mathbf{x},t) - f_i(\mathbf{x},t) = -\frac{1}{\tau}\left[f_i(\mathbf{x},t) - f_i^{eq}(\mathbf{x},t)\right] \\
&\text{MRT}: f_i{}^*(\mathbf{x},t) - f_i(\mathbf{x},t) = -\left(\mathbf{M}^{-1}\mathbf{S}\mathbf{M}\right)\left[f_i(\mathbf{x},t) - f_i^{eq}(\mathbf{x},t)\right] \\
&\text{CLBM}: f_i{}^*(\mathbf{x},t) - f_i(\mathbf{x},t) = -\left(\mathbf{M}^{-1}\mathbf{N}^{-1}\mathbf{S}\mathbf{N}\mathbf{M}\right)\left[f_i(\mathbf{x},t) - f_i^{eq}(\mathbf{x},t)\right]
\end{aligned} \tag{3}$$

The macroscopic variables such as density ρ and velocity $\mathbf{u}$ are computed using the moments of f:

$$\begin{aligned}
\rho &= \sum_{i=0}^{q-1} f_i \\
\rho\mathbf{u} &= \sum_{i=0}^{q-1} f_i \mathbf{e_i}
\end{aligned} \tag{4}$$

In JAX-LaB, for BGK and MRT collision models, the external forces are incorporated into the distribution function using the Exact Difference Method (EDM) (Kupershtokh et al., 2009), which modifies the post-collision distribution function using a difference of equilibrium distribution functions computed using the momentum before and after the action of force $\mathbf{F}$:

$$\begin{aligned}
\Delta\mathbf{u} &= \frac{\mathbf{F}\Delta t}{\rho} \\
\mathbf{F_m} &= f_i^{eq}(\rho, \mathbf{u} + \Delta\mathbf{u}) - f_i^{eq}(\rho, \mathbf{u}) \\
f_i{}^*(\mathbf{x},t) &= f_i{}^*(\mathbf{x},t) + \mathbf{F_m}
\end{aligned} \tag{5}$$

In this case, the macroscopic velocity is computed as:

$$\rho\mathbf{u} = \sum_i f_i \mathbf{e}_i + \frac{\mathbf{F}\Delta t}{2} \tag{6}$$

Although EDM is compatible with any collision operator as it is directly derived from the Boltzmann equation (Kupershtokh et al., 2009), for the cascaded collision model, a consistent forcing scheme based on second-order trapezoidal scheme proposed by Fei and Luo (2017) is used for enhanced performance, which introduces an additional forcing term to the central moments:

$$\begin{aligned}
f_i{}^*(x,t) - f_i(x,t) &= \mathbf{M}^{-1}\mathbf{N}^{-1}\left(-\mathbf{S}\mathbf{N}\mathbf{M}\left[f(\mathbf{x},t) - f_i^{eq}(\mathbf{x},t)\right] + (\mathbf{I} - \mathbf{S}/2)\mathbf{C}\right) \\
\mathbf{C} &= \mathbf{N}\mathbf{M}\mathbf{R}_i \\
\mathbf{R}_i &= \frac{\mathbf{F}}{\rho}\frac{(\mathbf{e}_i - \mathbf{u})}{c_s^2} f_i^{eq}
\end{aligned} \tag{7}$$

Similar to EDM, the macroscopic velocity is computed as:

$$\rho\mathbf{u} = \sum_i f_i \mathbf{e}_i + \frac{\mathbf{F}\Delta t}{2} \tag{8}$$

2.2. Multiphase and Multicomponent Flows

For multicomponent flows, the evolution of distribution f for the k-th component is obtained using LBE:

$$f_i^k(x + \mathbf{e_i}\Delta t, t + \Delta t) - f_i^k(x,t) = \Omega_i^k \quad i = 0, 1 \ldots q-1 \tag{9}$$

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

The corresponding fluid density and velocity are obtained using Equation 4:

$$\rho^k(\mathbf{x},t) = \sum_i f_i^k(\mathbf{x},t)$$
$$\mathbf{u}^k(\mathbf{x},t) = \sum_i f_i^k(\mathbf{x},t)\mathbf{e}_i/\rho^k(\mathbf{x},t) \tag{10}$$

The interactions among phases and components are modeled using the Shan-Chen pseudopotential method (Shan & Chen, 1993), chosen for several key advantages: (a) it is simple and computationally efficient, relying on a localized interaction force to induce phase separation, eliminating the need for explicit interface tracking (Coon et al., 2014; Gong & Cheng, 2012) or evolution of a separate order-parameter field (which is used in the free-energy method (Swift et al., 1996)); (b) it naturally supports both single-component and multicomponent multiphase systems unlike the color-gradient method (Gunstensen et al., 1991) which is limited to binary fluid mixtures; (c) it accommodates a broader range of density ratios than the color-gradient approach; (d) it preserves Galilean invariance, which the free-energy method does not; and (e) it allows for the use of arbitrary equations of state, whereas the free-energy method has primarily been demonstrated with the van der Waals EOS (Gong & Cheng, 2012). Despite these strengths, the Shan-Chen method has some limitations. It typically produces diffuse interfaces spanning several voxels, in contrast to the sharp, well-defined interfaces generated by the color-gradient method (Chen et al., 2019). Additionally, the Shan-Chen approach requires tuning the pressure modification parameter κ (Section 2.3) to indirectly control interfacial tension, while surface tension can be directly specified in the color-gradient model.

The force $F_{f-f}^{k,k'}$ represents the interaction between k and k'th fluid components and it is modeled as:

$$F_{f-f}^{k,k'}(\mathbf{x},t) = -A_{kk'}\nabla U^{k'}(\mathbf{x},t) - (1 - A_{kk'})\, g_{kk'}\psi^k(\mathbf{x},t)\nabla\psi^{k'}(\mathbf{x},t) \tag{11}$$

Equation 11 is the weighted sum of forces obtained from two different forcing schemes: the original Shan-Chen forcing scheme (Shan & Chen, 1993) and the Zhang-Chen forcing scheme (R. Zhang & Chen, 2003). This generalized approximation of the interaction force is widely used in the literature as it can model systems with a broader range of density ratios compared to each of the individual schemes (Gong & Cheng, 2012; Hu et al., 2014; Kupershtokh et al., 2009; Qin et al., 2022).

To model the behavior of non-ideal fluids, pressures obtained from a given equation of state (EOS) are used to compute the pseudopotentials used in Equation 11 (Hu et al., 2014; Kupershtokh et al., 2009; Yuan & Schaefer, 2006):

$$\psi^k(\rho) = \sqrt{\frac{2\left(\alpha^k p_{EOS}^k - c_s^2\rho^k\right)}{g_{kk}}}$$
$$U^k(\mathbf{x},t) = \alpha^k p_{EOS}^k(\mathbf{x},t) - c_s^2\rho^k(\mathbf{x},t) \tag{12}$$

These equations of state are implemented under a separate EOS class in code (src/eos.py), which can be appended by the user to add other equations of states (for instance, cross-over EOS to model supercritical fluids (Ashirbekov et al., 2021)). To run a multiphase simulation, an EOS object is passed as an argument to the consolidated multiphase parent class, and it is subsequently used by the pseudopotential computation function. This approach provides a convenient and elegant solution to incorporate an arbitrary EOS, as the EOS object can be easily replaced by another to use a different equation of state without making any modification to the multiphase code. The interaction strength $g_{kk'}$ controls the miscibility between different fluids. The intra-component strength g_{kk} is set as -1 to ensure that the term inside the square root stays positive (Yuan & Schaefer, 2006). The total force for the kth component is obtained by summing over all $F^{k,k'}$. The gradients of pseudopotentials ψ and U are approximated using a weighted sum of their values at neighboring lattice sites:

$$\begin{aligned}\mathbf{F}^k_{f-f} &= A_{kk}\,\mathbf{F}^k_{ZC} + (1 - A_{kk})\,\mathbf{F}^k_{SC} \\ \mathbf{F}^k_{SC} &= -\psi^k(\mathbf{x}) \sum_{k'=1}^{n_C} g_{kk'} \sum_{\mathbf{x}'} G(\mathbf{x},\mathbf{x}')\psi^{k'}(\mathbf{x}')\,(\mathbf{x}' - \mathbf{x}) \\ \mathbf{F}^k_{ZC} &= -\sum_{\mathbf{x}'} G(\mathbf{x},\mathbf{x}')\,U^k(\mathbf{x}')\,(\mathbf{x}' - \mathbf{x})\end{aligned} \tag{13}$$

By default, the Green's function $G(\mathbf{x},\mathbf{x}')$ specifies non-zero values only for nearest neighbor sites. However, users can specify non-zero values for next-nearest neighbors to achieve higher-order isotropy for the gradient approximation (Coon et al., 2014; Qin et al., 2022; Sbragaglia et al., 2007).

2.3. Interfacial Tension Control

In the Shan-Chen pseudopotential model, surface tension and density ratio are intrinsically linked, making it impossible to adjust one without affecting the other (Shan & Chen, 1993). Although adjusting the modification parameter α^k and the weighting coefficients $A_{kk'}$ allows for some control over surface tension, the tunable range is limited (typically 2–3 times of unadjusted value) without compromising the ability to reproduce the correct coexistence densities (Hu et al., 2014). To overcome this constraint, JAX-LaB implements the pressure tensor modification approach developed by Q. Li and Luo (2013) to tune the surface tension without changing the mechanical stability of the multiphase system. In this method, the MRT collision step described in Equation 3 is modified to incorporate a source term $\mathbf{C}$:

$$f^k_i(x + \mathbf{e}_i\Delta t, t + \Delta t) - f^k_i(x + \mathbf{e}_i\Delta t, t) = -\mathbf{M}^{-1}\left[(\mathbf{SM})\left(f^k_i - f^{k,eq}_i\right) - F_m\right] + \mathbf{C} \tag{14}$$

For D2Q9 lattice, $\mathbf{C}$ is given as (Q. Li & Luo, 2013):

$$\mathbf{C} = [0, 1.5s_e(Q_{xx} + Q_{yy}),\ -1.5s_\eta(Q_{xx} + Q_{yy}), 0, 0, 0, 0,\ -s_\nu(Q_{xx} - Q_{yy}),\ -s_\nu Q_{xy}]^T \tag{15}$$

where the terms Q_{xx}, Q_{yy} and Q_{xy} are evaluated using (Hu et al., 2014):

$$\begin{aligned}\mathbf{Q} = \kappa\Bigg[&(1 - A_{kk})\psi(\mathbf{x}) \sum_{i=0}^{q-1} w\left(|\mathbf{e}_\alpha|^2\right)\left(\psi^k(\mathbf{x} + \mathbf{e}_\alpha) - \psi^k(\mathbf{x})\right)\mathbf{e}_\alpha\mathbf{e}_\alpha + \\ &\frac{A_{kk}}{2} \sum_{i=0}^{q-1} w\left(|\mathbf{e}_\alpha|^2\right)\left(\psi^k(\mathbf{x} + \mathbf{e}_\alpha)^2 - \psi^k(\mathbf{x})^2\right)\mathbf{e}_\alpha\mathbf{e}_\alpha\Bigg]\end{aligned} \tag{16}$$

$\mathbf{Q}$ is based on the discrete pressure tensor formulation for the pseudopotential method (Shan, 2008). The modified pressure tensor obtained after the addition of the source term causes the surface tension to scale with $1 - \kappa$, reaching its maximum magnitude when this term equals unity. The formulation of source $\mathbf{C}$ for D3Q19 lattice can be found in Zhu et al. (2024).

2.4. Wettability Control

In JAX-LaB, solid-fluid wettability is implemented using the improved virtual density scheme proposed by Q. Li et al. (2019). This method simulates wettability by locally modifying density values at solid nodes:

$$\rho_s = \begin{cases}\phi\rho_{\text{ave}}(x_s), & \phi \geq 1, \quad \theta \leq 90^\circ \\ \rho_{\text{ave}}(x_s) - \Delta\rho, & \Delta\rho \geq 0, \quad \theta > 90^\circ\end{cases} \tag{17}$$

where ρ_s is the density at solid nodes x_s, ϕ & $\Delta\rho$ are control parameters and ρ_{ave} is a weighted average of neighboring fluid densities (Q. Li et al., 2019). To ensure numerical stability, a limiter constrains the solid node densities from Equation 17 to lie within the problem's physical density bounds. By employing spatially varying solid densities rather than a uniform global value, this scheme retains the strengths of the geometric contact angle

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

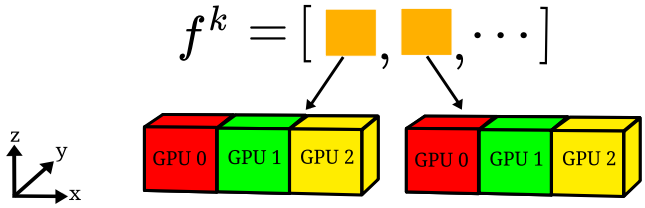


Figure 1. Illustration of computational sharding strategy and data structure employed to store JAX arrays and distribute them equally across multiple devices. Arrays are sliced along the *x*-axis and then divided across multiple devices. These sharded arrays are stored together in a list to form a pytree, with each JAX array storing values for one component.

method (Ding & Spelt, 2007) without the need to compute characteristic directions, which can be computationally expensive, especially in three dimensions (Q. Li et al., 2019). In doing so, it overcomes major drawbacks of traditional Shan-Chen wettability mod'els, namely, large spurious currents at high density ratios, limited control over contact angles at high density ratios, and the emergence of non-physical interfacial films whose thickness is tied to density ratio rather than wettability (Hu et al., 2016; Q. Li et al., 2019), while still achieving a broad spectrum of contact angles by tuning ϕ, ρ_{ave}, and $\Delta\rho$.

2.5. Model Implementation

JAX-LaB leverages JAX (Bradbury et al., 2018), a high-performance hardware-agnostic library from Google that breaks down functions into efficient primitives and uses them to generate machine code for accelerators like GPUs and Tensor Processing Units (TPUs). The functions are traced with abstract arguments (encoding shape and type) on the first call, and the optimized code generated is reused directly for subsequent calls with matching inputs, enabling significant performance gains (Bradbury et al., 2018). As LBM computations rely on fixed-shape, fixed-precision arrays, JAX-LaB leverages an optimized implementation of the LB step (Equation 2) for significant performance enhancement.

2.6. Data Structure Design

In XLB (Ataei & Salehipour, 2024), GPU-accessible arrays are defined as JAX arrays with dimensions (*x*, *y*, cardinality) in 2D or (*x*, *y*, *z*, cardinality) in 3D. These arrays are distributed across multiple GPUs using a computational sharding strategy that slices the array along the *x*-axis and assigns each slice to a GPU. To balance the load between different GPUs, nx is adjusted to be divisible by the number of GPUs. Furthermore, initialization is restricted to the portion of the array accessible by each GPU, which is crucial for arrays that are too large to be initialized on a single GPU.

JAX-LaB extends these concepts to multicomponent flows by using *pytrees*, a list-based data structure introduced by JAX to store all computational parameters. During initialization, a JAX array is created to store distribution values for each component (using the previously described computational sharding strategy) and collected into a list to form a pytree (Figure 1). To carry out computations, the nodal formulations of various quantities, such as the interaction force, pressure tensor modification, and others, are expressed as equivalent tensor operations. These operations are then applied to the pytrees, producing a new pytree of JAX arrays for all components. This approach is demonstrated using the original Shan-Chen force in Figure 2, and it is used for all computations, leading to a concise formulation that does not change with the number of components. The data structure is also used for storing various floating point values (for instance, fluid-fluid interaction strengths g_k in Figure 2) as it can be passed to functions as a pytree to perform computation.

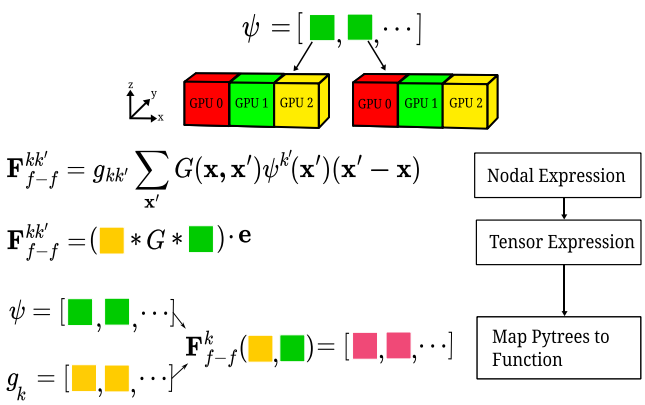


Figure 2. Implementation of Shan-Chen force using pytrees in JAX-LaB. The colored squares indicate the different values of pytree, which can be either a floating-point value or a JAX array.

3. Benchmarks and Applications

To validate JAX-LaB's suitability for modeling real-world multiphase systems, we tested JAX-LaB against several established analytical benchmarks. These benchmarks examine its ability to (a) reproduce thermodynamically consistent densities per the equation of state (Section 3.1), (b) adjust fluid surface tension without altering the system's density ratio (Section 3.2), and (c) incorporate wettability in both single-component and multicomponent multiphase configurations (Sections 3.3.1, 3.3.2, 3.3.3). The maximum achievable density ratios for each EOS are dependent on the EOS parameters considered, and in the subsequent section, the performance is evaluated for one set of parameters, which are taken from Yuan and Schaefer (2006)'s work. The src/examples directory in the repository contains scripts for reproducing the benchmarks and applications discussed in this section. The examples presented in Section 3.4 highlight the versatility of JAX-LaB for modeling porous media flows.

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

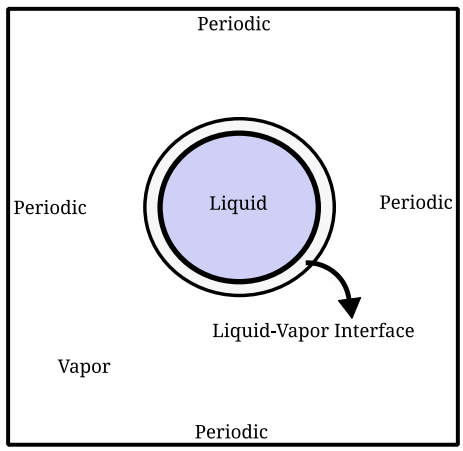


Figure 3. Schematic diagram for a droplet suspended in vapor.

3.1. Thermodynamic Consistency

Coexistence densities are the densities of different phases, such as liquid and vapor, in equilibrium at a specific temperature and pressure. 2D and 3D simulations of a static liquid droplet suspended in its vapor (Figure 3) were performed in JAX-LaB to calculate coexistence densities and verify thermodynamic consistency. In these tests, the parameters for each of the equations of state are taken from Yuan and Schaefer (2006). The 2D and 3D simulations were domains of sizes 200^2 and 200^3 lattice units, respectively, with a droplet of radius 30 lattice units placed at the domain centers. The initial density profile of the domain was specified as (Qin et al., 2022):

$$\rho = \frac{\rho_l - \rho_g}{2} - \frac{\rho_l - \rho_g}{2} \cdot \tanh\left(2 * \frac{(d-r)}{w}\right) \tag{18}$$

where d is the Euclidean distance of all points in the domain from the droplet center and w is the width of the interface, set as 5 lattice units in the present simulation. The simulations were performed using D2Q9 or D3Q19 lattice with MRT collision model (McCracken & Abraham, 2005) and run until steady state was achieved. The values of α^k and $A_k k'$ in Equation 13 were adjusted for each EOS to ensure that the mechanical stability condition was satisfied. The coexistence densities obtained from these lattice Boltzmann simulations are compared against the values obtained from Maxwell's construction. In the simulation, five equations of state are considered: Carnahan-Starling (CS), Peng-Robinson (PR), Redlich-Kwong (RK), Redlich-Kwong-Soave (RKS), and Van der Waals (VdW). The coexistence densities (scaled by critical density ρ_c to get reduced densities ρ_r) for these EOS are evaluated at different reduced temperatures Tr, and the results are shown in Figure 4. The parameters used for each equation of state are taken from the works of Yuan and Schaefer (2006). Figure 4 clearly demonstrates that the coexistence densities obtained from simulation closely match the expected values for a large range of reduced temperature T_r, achieving maximum density ratios of 8.439×10^7, 4.84×10^5, 2.74×10^5, 6.77×10^4 and 5.4×10^4 for CS, PR, RK, RKS, and VdW equation of states respectively with root mean square errors of 2.6×10^{-3}, 4.2×10^{-2}, 3.9×10^{-2}, 4×10^{-2} and 5.1×10^{-2} respectively.

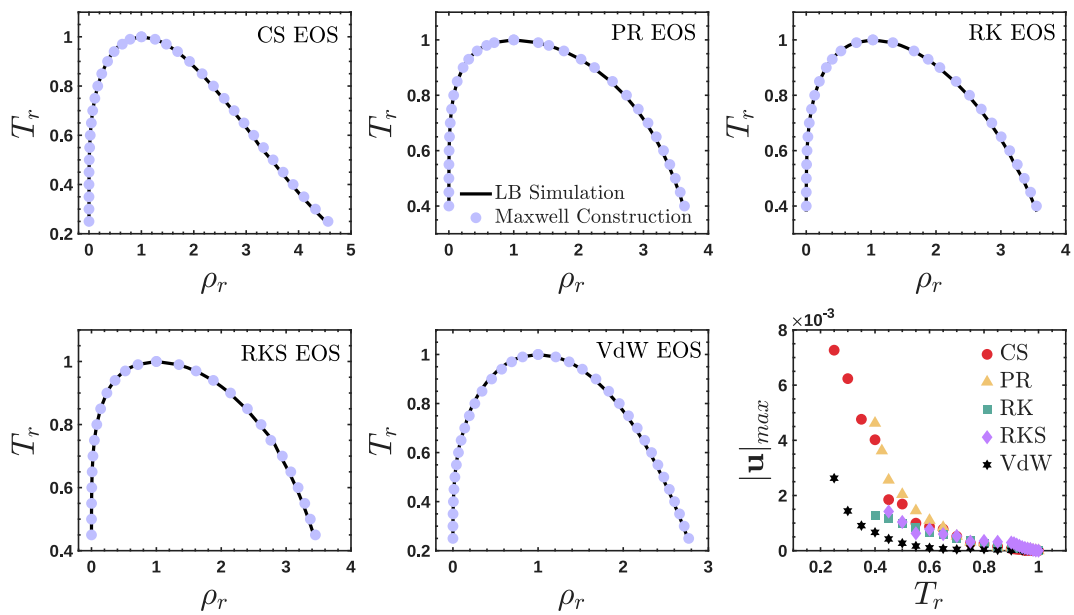


Figure 4. Comparison of coexistence densities versus reduced temperature derived from Lattice Boltzmann simulations with predictions from the Maxwell construction for various equations of state (EOS): (a) Carnahan-Starling (CS) (b) Peng-Robinson (PR) (c) Redlich-Kwong (RK), (d) Redlich-Kwong-Soave (RKS), and (e) van der Waals (VdW). (f) Peak spurious current magnitudes across various temperatures for different equations of state. All values are in lattice units.

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

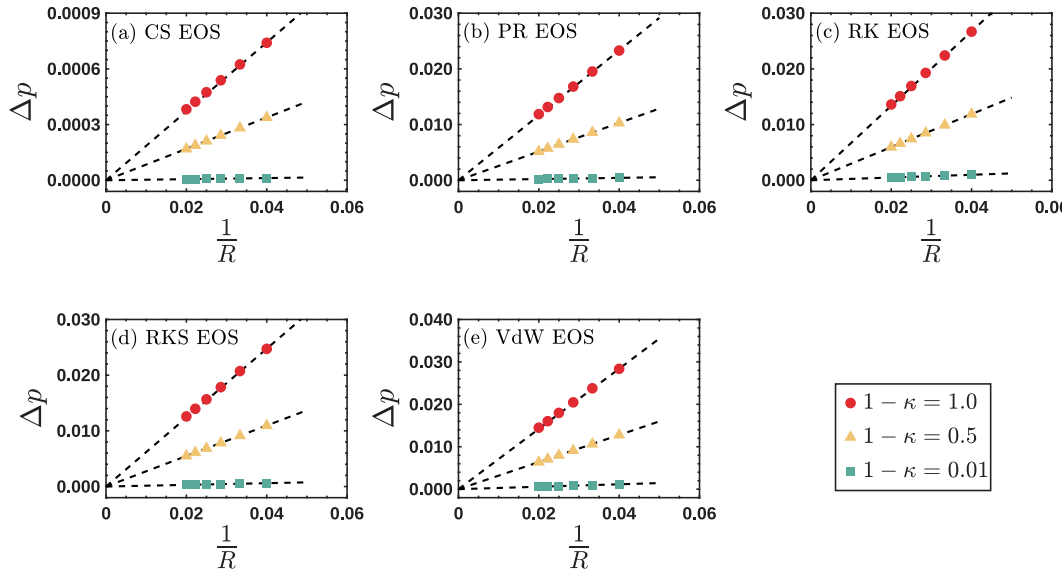


Figure 5. The relationship between pressure differential and droplet curvature ($1/R$) plotted for various values of κ, shown separately for (a) Carnahan-Starling (CS), (b) Peng-Robinson (PR), (c) Redlich-Kwong (RK), (d) Redlich-Kwong-Soave (RKS), and (e) van der Waals (VdW) equation of state at $\tau_v = 1.0$ and $T_r = 0.8$. All values are in lattice units.

3.2. Laplace Law

To validate the numerical implementation described in Section 2.2 against Laplace's law, 2D and 3D simulations of a circular droplet are conducted using the D2Q9 and D3Q19 lattices, respectively. Laplace's law defines the relationship between the pressure difference inside and outside the droplet (Δp) and inverse droplet radius ($1/R$):

$$\Delta p = \frac{\sigma}{R} \tag{19}$$

where σ is the surface tension constant. The simulations are performed in a domain size of 200^2 for 2D and 200^3 for 3D, with periodic boundary conditions applied throughout. A circular droplet is placed at the center, with radius varying between 25 and 50 lattice units with increments of 5 lattice units (Figure 3). The reduced temperature T_r is set as 0.8 for all EOS considered, and the density field is initialized using Equation 18 with width $w = 3$ lattice units. To adjust surface tension, the MRT collision model is used. Similar to the previous case, the parameters for each of equation of state are taken from Yuan and Schaefer (2006) and the values of α^k and $A_{kk'}$ in Equation 13 are adjusted for each EOS to ensure that the mechanical stability condition is satisfied. Figure 5 illustrates that the computed pressure difference across the interface is proportional to the inverse radius of the droplet for all equations of state considered, thus verifying the Laplace law.

Moreover, the coexistence densities obtained for different surface tension adjustment coefficients κ for the identical testing case are tabulated in Table 1. The results reveal that the computed densities for all EOS do not vary with κ and are in close agreement with the predictions from the Maxwell construction. Furthermore, the surface tension values listed in Table 2 indicate that surface tension can be modified by factors of 110, 58, 100, 92, and 115 for the CS, PR, RK, RKS, and VdW equations of state, respectively, without altering the density ratio proving that the pressure tensor modification methods allows independent adjustment of surface tension without affecting the liquid-vapor coexistence densities unlike the original Shan-Chen method.

3.3. Capillary Flows

3.3.1. Capillary Rise in a Tube

During spontaneous imbibition in channels, the wetting fluid advances by displacing the non-wetting phase, and this process can be classified into liquid-liquid or gas-liquid drainage. Gas-liquid drainage in particular has been the focus of extensive study because of its importance in applications such as fracturing-fluid invasion in

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 1
Comparison Between Simulated and Maxwell Coexistence Densities for Different Surface Tension Adjustment Coefficient κ at $T_r = 0.8$

ρ_g/ρ_l				
EOS	$1-\kappa = 1.0$	$1-\kappa = 0.5$	$1-\kappa = 0.01$	Maxwell Construction
CS	0.0217/0.3091	0.0217/0.3088	0.0217/0.3072	0.0217/0.3071
PR	0.1971/7.2063	0.1971/7.2063	0.1971/7.2063	0.1970/7.2040
RK	0.3425/6.6734	0.3426/6.6734	0.3427/6.6734	0.3428/6.6252
RKS	0.2210/7.0692	0.2210/7.0695	0.2210/7.0697	0.2208/7.0693
VdW	0.8389/6.7635	0.8381/6.7605	0.8386/6.7681	0.8388/6.7644

Note. All values are in lattice units.

unconventional reservoirs, CO_2 storage, and aqueous-phase transport in underground gas storage reservoirs (Y. Yang et al., 2023). A 2D single-component simulation is performed using the MRT collision model and D2Q9 lattice on a 600×100 domain. A channel with length (L) of 300 lattice units and width of 28 lattice units is set at the position of $150 < x < 450$, with static contact angle $\theta = 19°$. No-slip (bounceback) boundary condition is applied at the channels, while a periodic boundary condition is applied at all other peripheral nodes. The simulation is initialized with positions $x < 150$ occupied by liquid (wetting phase), and the rest occupied by the gas phase. VanderWaal equation of state with parameters $a = 9/49$, $b = 2/21$, and $R = 1$ is used to model the single-component multiphase system at reduced temperature $T_r = 0.7$. The surface tension adjustment factor is set as $\kappa = 0.5$. Considering capillary, inertia, and viscous forces, the meniscus position l at time t for a fluid undergoing horizontal imbibition can be determined using the following equation (X. Yang et al., 2025):

$$\frac{2\sigma\cos(\theta)}{H} - \left[\rho_l l + (L-l)\rho_v\right]\frac{d^2 l}{dt^2} - \left[\rho_l l + (L-l)\rho_v\right]\left(\frac{dl}{dt}\right)^2 - \frac{12}{H^2}\left[\mu_l l + (L-l)\mu_v\right] = 0 \tag{20}$$

where σ, H, ρ, μ are the surface tension coefficient, width of the capillary, density, and dynamic viscosity, respectively (Figure 6a; subscripts l and v denote respective values for liquid and vapor phase.) Equation 20 can be simplified by ignoring gas-phase viscosity, inertial forces, and dynamic wetting effects to get the original Lucas-Washburn equation (Washburn, 1921). Although the Lucas-Washburn equation closes matches macroscopic experimental results, at micro-scale and nano-scale, various fluid properties such as viscosity and contact angle are not constant and hence, the Lucas-Washburn equation leads to an overprediction of meniscus position as confirmed in several experimental studies (Heshmati & Piri, 2014; Siebold et al., 2000). Among these parameters, the most noticeable impact on meniscus height prediction comes from using a static contact angle, instead of a dynamic contact angle (Hamraoui et al., 2000). Thus, in this paper, Equation 20 is solved using fourth-order explicit Runge-Kutta scheme with initial conditions $l(0) = 0$ and $\frac{dl}{dt}|_{t=0} = 0$ (Wolf et al., 2010) and the dynamic contact angle θ_d is measured during the simulation using the height of the bent liquid surface h_m (X. Yang et al., 2025):

$$\theta_d = \cos^{-1}\left(\frac{4h_m H}{H^2 + 4h_m^2}\right) \tag{21}$$

Figure 6b plots the meniscus positions obtained from the LB simulation against those obtained from the analytical expression, showing close agreement between the two.

Table 2
Surface Tension Values Obtained Using the Present Method for Different Surface Tension Adjustment Coefficient κ at $T_r = 0.8$

Surface tension			
EOS	$1-\kappa = 1.0$	$1-\kappa = 0.5$	$1-\kappa = 0.01$
CS	0.0186	0.0085	0.00017
PR	0.5831	0.2574	0.0101
RK	0.6684	0.2969	0.00681
RKS	0.6189	0.2738	0.00675
VdW	0.7099	0.3194	0.00617

Note. All values are in lattice units.

3.3.2. Drainage Regimes in a Channel

This section benchmarks the morphology of an invading non-wetting front (i.e., drainage) within a capillary channel for the case of immiscible displacement against the empirical model developed in Halpern and Gaver (1994). Depending on the dominant mechanism, the displacement can be classified as

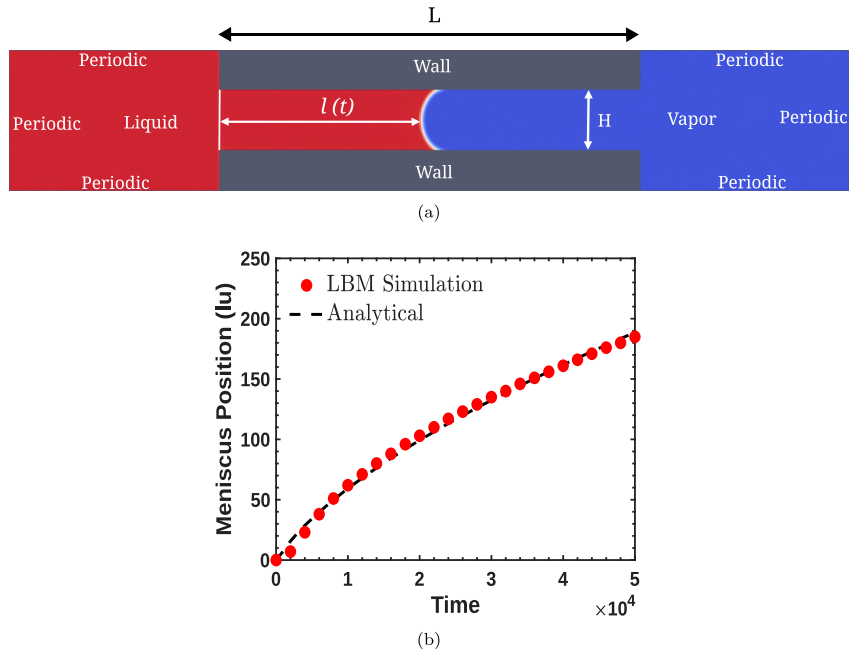


Figure 6. (a) Schematic diagram of capillary rise process. (b) Comparison between imbibition lengths obtained from simulation and analytical expression (in lattice units).

either capillary fingering, viscous fingering, or stable displacement, and the influence of capillary and viscous forces is quantified using two dimensionless numbers: the capillary number (Ca) and the viscosity ratio (M) (Lenormand et al., 1988). A 2D multicomponent fluid simulation is performed for two fluids in a channel using the D2Q9 lattice and MRT collision model. Component 1 occupies the left half of the channel, while component 2 occupies the right half. Initially, a small fraction of each fluid component is introduced into the bulk of the other to establish miscibility, with the exact value determined by the strength of inter-component interactions. It is crucial to ensure that each component maintains a non-zero density throughout the domain to prevent division by zero during velocity computation (Equation 4). In this benchmark, the fraction is fixed at 3%, following the approach of Kang et al. (2004). The intra-component interaction strength is set to a small negative value of -0.027, while the inter-component interaction strength is assigned a large positive value of 0.57. These values are tuned to minimize the thickness of the interface between component 1 and component 2, thereby promoting immiscibility. The channel inlet and outlets are set as periodic boundaries, while the channel walls are set as no-slip boundaries. A body force is applied along the channel to drive the fluids. The simulations are performed until the breakthrough of component 1 is achieved. The interaction forces are computed using the original Shan-Chen scheme by setting $A_{kk'} = 0$ in Equation 13, and the pseudopotential computation function in the code is modified to use the component density as pseudopotential $\left(\psi^k = \rho^k\right)$. The simulations are performed for the case of both fluids being neutrally-wet (contact angle = 90°, $\phi = 1.0$, and $\Delta\rho = 0.0$ in Equation 17). The surface tension coefficient used for computing Ca is determined using Laplace's law, as outlined in Section 3.2. The channel simulations are performed using a 500×75 lattice, and the prescribed force along the flow direction is varied to achieve various capillary numbers. The interface is defined at the location where both components have identical density, and the maximum finger width is determined using the ImageJ Fiji software (Schindelin et al., 2012). The relative width of the finger (finger width/channel width) obtained in each case is plotted as a function of Ca in Figure 7, and these results agree with the empirical model developed in Halpern and Gaver (1994)'s work.

3.3.3. Relative Permeability in Layered Cocurrent Flow

The cocurrent flow of two components in a channel is illustrated in Figure 8a. The central region contains the non-wetting phase, while the surrounding area is occupied by the wetting phase. The flow is driven by a body force, and the relative permeability (K_r) of each phase is determined as the ratio of its flow rate in cocurrent flow to its flow rate in a single-phase simulation:

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

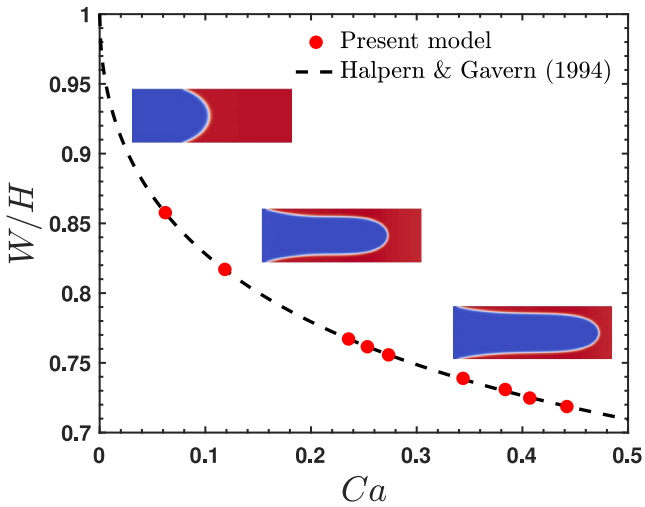


Figure 7. Comparison of the relative finger width (finger width normalized by channel width) obtained using the present method with the empirical relation from Halpern and Gaver (1994) for density ratio of 1.0. The inset figure illustrates finger shapes at different *Ca* values, showing stable displacement at low *Ca*.

$$K_{r,w} = \frac{\int_{|y|=0}^{a} u_{nw} dy}{\int_{|y|=0}^{H} u_{nw} dy}$$
$$K_{r,w} = \frac{\int_{|y|=a}^{H} u_{w} dy}{\int_{|y|=0}^{H} u_{w} dy} \quad (22)$$

The steady state expression for the relative permeability (K_r) can be obtained analytically, assuming laminar flow and a sharp interface between the components (Porter et al., 2012):

$$K_{r,w} = \frac{1}{2} S_w^2 (3 - S_w)$$
$$K_{r,nw} = (1 - S_w)\left[\frac{3}{2}M + (1 - S_w)^2\left(1 - \frac{3}{2}M\right)\right] \quad (23)$$

where S_w is the wetting phase saturation, defined as the ratio of wetting phase width to channel width (a/H), and M is the dynamic viscosity ratio.

A 2D multicomponent fluid simulation is performed for flow in a channel using D2Q9 lattice and the MRT collision model. The channel walls are set as no-slip boundaries, while periodic boundary conditions were applied at the inlet and outlet. Using density as pseudopotential $\left(\psi^k = \rho^k\right)$, the inter- and intra-component interaction forces are tuned to ensure that the interface thickness between two components is less than or equal to 4 lattice units. Figure 8 shows a close agreement between the simulated relative permeability values with Equation 23 analytical results for viscosity ratio $M = 1$.

3.4. Porous Media Flows

3.4.1. Permeability Estimation for a Synthetic Porous Media

JAX-LaB is utilized to determine the permeability of a given porous geometry. The geometry used here is obtained from the Digital Rocks Portal (Santos et al., 2025; Santos, Pyrcz, & Prodanović, 2022) (geometry: 374_05_03_256) and represents a 256^3 randomly placed overlapping sphere pack with an embedded fracture, having porosity 0.1962. To compute permeability, a pressure gradient of 0.01 in lattice units is applied along the flow direction (*x*-direction), while a no-slip boundary condition is imposed along the solid nodes of the geometry, *y* and *z* direction faces. The domain is initialized with zero velocity everywhere and a linear pressure profile along the *x* direction. The simulation is run until a steady state is achieved (50,000 lattice timesteps). Plugging the values of the steady-state mean velocity obtained from simulation and the applied pressure gradient into Darcy's law, the

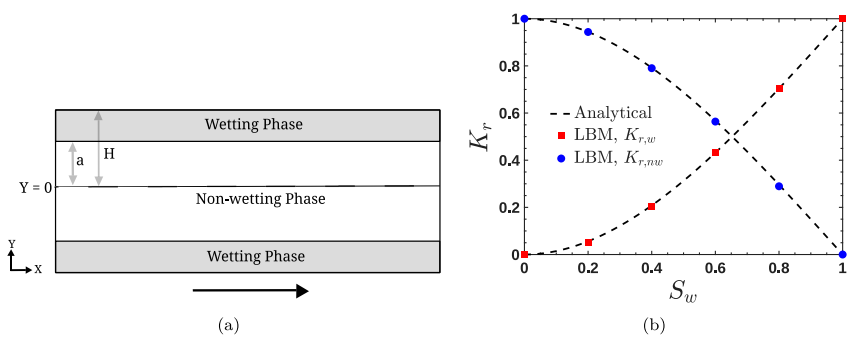


Figure 8. (a) Schematic of two-phase cocurrent flow in a channel. (b) Comparison of simulated and analytical relative permeability curves for the wetting phase ($K_{r,w}$) and non-wetting phase ($K_{r,nw}$) in cocurrent flow with a viscosity ratio of $M = 1.0$.

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

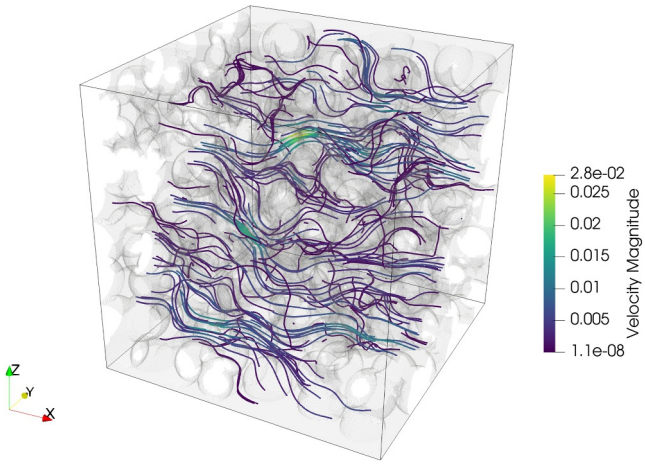


Figure 9. Streamlines for the steady-state solution in a domain comprised of randomly placed overlapping spheres and an embedded synthetic fracture; the geometry was obtained from the Digital Rocks Portal (Santos et al., 2025).

permeability of the model is determined to be 2.003 Darcy, which falls within the expected range of 2–3 Darcy (Santos et al., 2025) (Figure 9).

3.4.2. Drainage Simulation in Fontainebleau Sandstone

This section models a two-component flow of water and supercritical CO_2 through a Fontainebleau sandstone obtained from the Digital Rocks portal (Santos et al., 2025) and showcases the use of JAX-LaB for modeling multicomponent, multiphase flow in porous media, with applications in CO_2 sequestration (Ashirbekov et al., 2021), enhanced oil recovery (Xu et al., 2020), evaporation (Fei et al., 2022) and beyond. It also describes the procedure used for setting the inter-component strengths and wettability parameters in multi-component simulations. The parameters used in Equations 13, 15 and 17, which govern inter-fluid interaction and wettability, are calibrated using a two-component droplet simulation (150^3 domain, D3Q19 lattice and MRT collision model). The simulation is initialized with a central water droplet surrounded by supercritical CO_2. The initial density distribution and mutual solubility of the fluids are tabulated in table 3, and are set following Tang et al. (2024), corresponding to fluid properties at 20 MPa and 200°C. We employ the Peng-Robinson equation of state in this supercritical regime because its predictions align well with experimental observations (Ashirbekov et al., 2021; Tang et al., 2024). The inter-fluid interaction coefficient $(g_{kk'})$ is chosen to be small and positive (weakly repulsive), while the intra-fluid coefficient (g_{kk}) is set to -1 (indicating strong cohesion), and the modification coefficient A_{kk} is set to 0. Relaxation times are selected to maintain a viscosity ratio of $\frac{\tau_{H_2O}}{\tau_{CO_2}} = 1.43$, representing the viscosity contrast between water and $scCO_2$, assuming identical viscosities for liquid and vapor states of water. Surface tension of this system is calculated using Laplace's law (modification coefficient $\kappa_{CO_2} = 0.2$). The steady-state densities obtained from the droplet simulation lead to a density ratio of 506. The wettability of the system is characterized by the steady-state contact angle formed by a water droplet suspended in $scCO_2$ on a spherical surface. Parameters ϕ and $\Delta\rho$ are then tuned to produce a contact angle of 30°, following Chen et al. (2019). The consolidated results from these simulations, used for parameter estimation, are presented in Figures 10a and 10b. These parameters are subsequently applied in the pore-scale simulations.

The top, bottom, front, and back faces of the 256^3 Fontainebleau sandstone (with porosity of 0.1938) domain are set as periodic boundaries, and the solid nodes in the domain are set as no-slip boundaries using the bounceback scheme. Buffer layers are added to the domain at the inlet and outlet to prevent end effects at the domain boundaries. Initially, $scCO_2$ is assumed to fully occupy the inlet buffer layer, and it is driven into the system using body force. For the outlet, a convective boundary condition (Lou et al., 2013) based on average velocity is used. The thickness of the input buffer layer is set such that the amount of $scCO_2$ present in it is enough to fill all the pores in the domain. The snapshots from the drainage simulation at various stages of saturation are shown in Figure 11.

3.4.3. Characteristic Curves for a Sphere Pack

This section illustrates the application of JAX-LaB for generating capillary pressure (P_c) v/s wetting component saturation (S_w) characteristic curves for a 256^3 synthetic sphere pack (geometry: 374_05_03_256) with a porosity of 0.381 sourced from the Digital Rocks Portal (Santos et al., 2025). It highlights JAX-LaB's capabilities for simulating multicomponent, multiphase flow in porous media, as well as its in situ measurement and visualization features that eliminate the need for post-processing. A 3D multiphase simulation is conducted using the BGK collision model, D3Q19 lattice, and two fluid components. Following the methodology of Galindo-Torres et al. (2016), both components are assigned the same density. Wettability effects are introduced by tuning the parameters in Equation 17 to yield a contact angle of 45° for the wetting component. The simulation domain is depicted in Figure 12a. Buffer layers, each 8 lattice units thick, are added near the inlet and outlet, with wall boundary conditions applied to all other faces

Table 3
Parameters of Peng-Robinson Equation of State for Carbon Dioxide and Water in Lattice Units From Tang et al. (2024)

Component	a	b	ω	R
Water	0.0204	0.09523	0.344	1
Carbon dioxide	0.01348	0.13385	0.22491	1

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

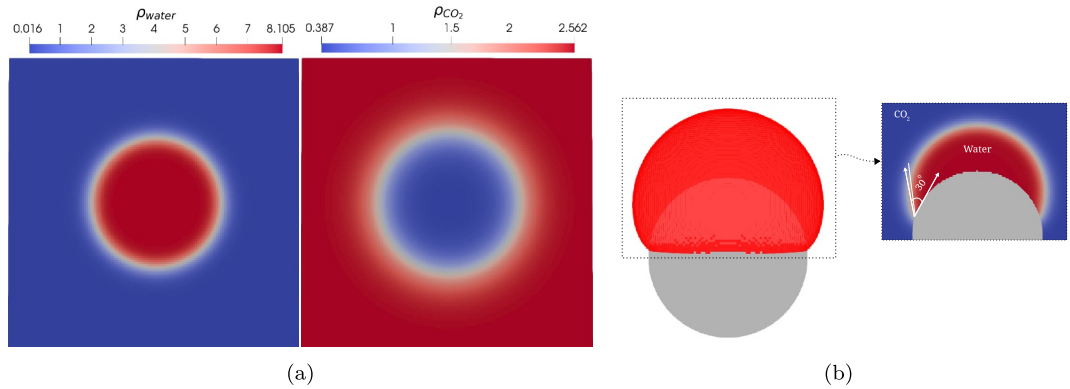


Figure 10. Three-dimensional multi-component droplet simulation (with density values shown in lattice units): (a) for determining fluid-fluid interaction strengths and (b) for evaluating fluid wettability parameters on a curved surface. A density threshold is used to isolate the droplet (red) from the background for visualization, and the contact angle formed on spherical geometry (gray) is measured at steady state. The density profiles are shown for a slice taken at droplet center ($y = 75$).

and the porous matrix. Constant pressure boundary condition is applied at the inlet and outlet to drive the fluids. Although flux boundary conditions have been employed in previous studies for capillary pressure-saturation measurements (Galindo-Torres et al., 2016; Porter et al., 2009), we adopt pressure boundary conditions here because they more closely mirror the experimental procedures used to derive characteristic curves (Porter et al., 2009) and enable efficient capture of complete drainage and imbibition behavior in a single simulation, starting from either a fully saturated or fully dry state. To obtain data points along the curve, P_c and S_w are computed after every 1,000 lattice timesteps, with each simulation running for a total of 250,000 timesteps. Since both components remain present throughout the domain, a threshold-based segmentation is applied: lattice sites where the non-wetting component density equals or exceeds that of the wetting component are classified as non-wetting (Galindo-Torres et al., 2016; Porter et al., 2009). P_c is computed using the average pressure of each component within its respective regions after segmentation, while S_w is calculated as the fraction of sites containing the wetting component. Fluid distributions within the porous domain are visualized in situ during the simulation using JAX arrays stored on GPU through the PhantomGaze library (Hennigh, 2023), enabling real-time monitoring without post-processing. Figure 12c demonstrates this capability by showing the evolution of the non-wetting component at selected timesteps during the drainage simulation.

4. Physics-Based Machine Learning

The native differentiability of XLB and its applications in scientific machine learning have been thoroughly discussed by Ataei and Salehipour (2024), who demonstrated that a unified computational graph for both forward and backward passes enables seamless integration of neural components within physical simulations. This framework allows machine learning models to be incorporated seamlessly at multiple stages of the simulation



Figure 11. Snapshots of invading component ($scCO_2$) distributions for simulation of drainage in a water-saturated Fontainebleau sandstone.

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

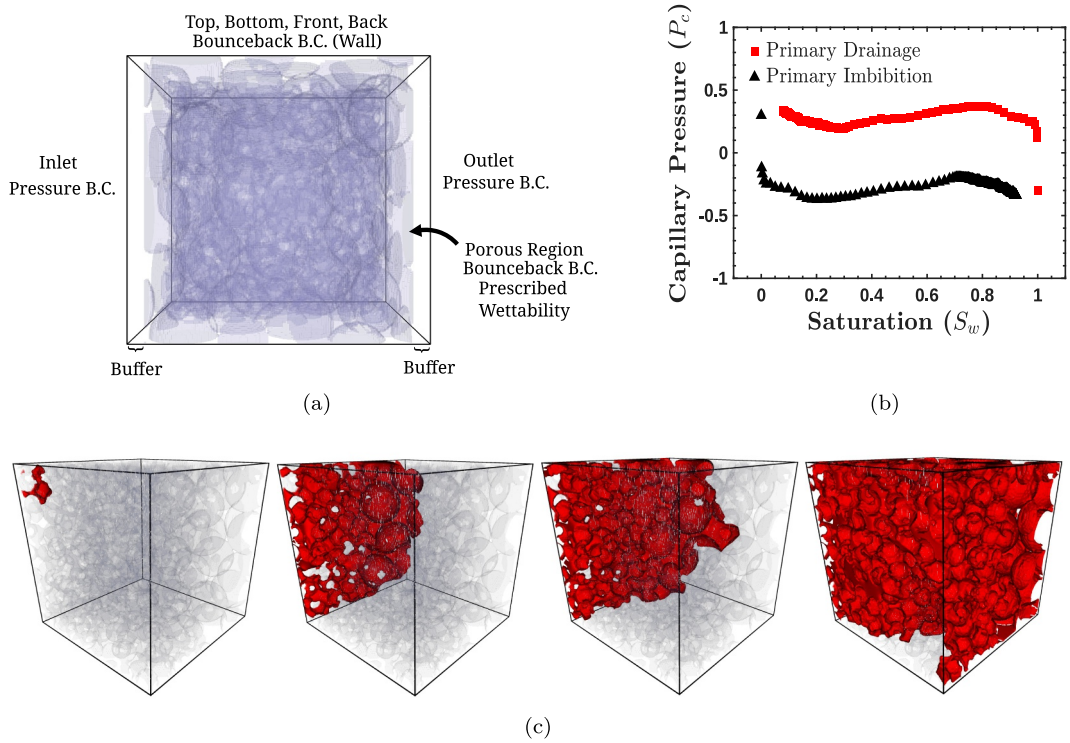


Figure 12. (a) Schematic of simulation domain used to obtain drainage/imbibition curves for a sphere pack. (b) Characteristic curves obtained from the drainage/imbibition process. (c) Distributions of non-wetting component during a primary drainage simulation for the synthetic sphere pack at various timesteps. The translucent gray region represents the solid phase, and the wetting component is not shown for visual clarity. The images are generated in situ from JAX arrays on GPU using the PhantomGaze library (Hennigh, 2023).

workflow. In this section, we demonstrate how JAX-LaB extends these capabilities to multiphase flow systems by addressing an inverse flow control problem, where automatic differentiation is employed during the neural network's backpropagation (Figure 13a). For this demonstration, we use the JAX-based Equinox library (Kidger & Garcia, 2021) to define and train the embedded neural network within the hybrid (physics plus NN) model. Specifically, the objective of the demonstration is to estimate an initial density field such that, after t_k timesteps of nonlinear fluid evolution (including intra and inter-phase interactions arising from Shan-Chen forces), the resulting density field approximates a droplet suspended in its vapor (henceforth referred to as the ground truth density). The simulation is performed on a 100×100 grid, employing the BGK collision model and the Van der Waals equation of state with parameters from Section 3.2, at a reduced temperature of $T_r = 0.8$. The corresponding liquid and vapor densities are $\rho_l = 6.76447$ and $\rho_g = 0.838834$, respectively. The ground-truth density field (ρ_{ground}) to be reproduced at $t = t_k$ is defined analogously to Equation 18:

$$\rho_{\text{ground}} = \frac{\rho_l - \rho_g}{2} - \frac{\rho_l - \rho_g}{2} \tanh\left(2\frac{d-r}{w}\right) \tag{24}$$

Here, $r = 25$ and $w = 4$. Periodic boundary conditions are applied in all directions, and the initial velocity field is set to zero. The initial density field required to reproduce this target or ground truth density distribution is obtained using a feedforward multi-layer perceptron with hidden layers of 512, 256, 256, and 512 neurons, respectively, as a representative example. Training is performed for 300 epochs using the adaptive moment estimation (ADAM) optimizer (Kingma & Ba, 2017) with a learning rate of 1×10^{-3}. For this optimization, we use the mean squared error (MSE) loss function, which is defined as:

$$\mathcal{L} = \frac{1}{nx \cdot ny} \sum \left(\sum_{i=1}^{q} f_i^{t_k} - \rho_{\text{ground}} \right)^2 \tag{25}$$

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

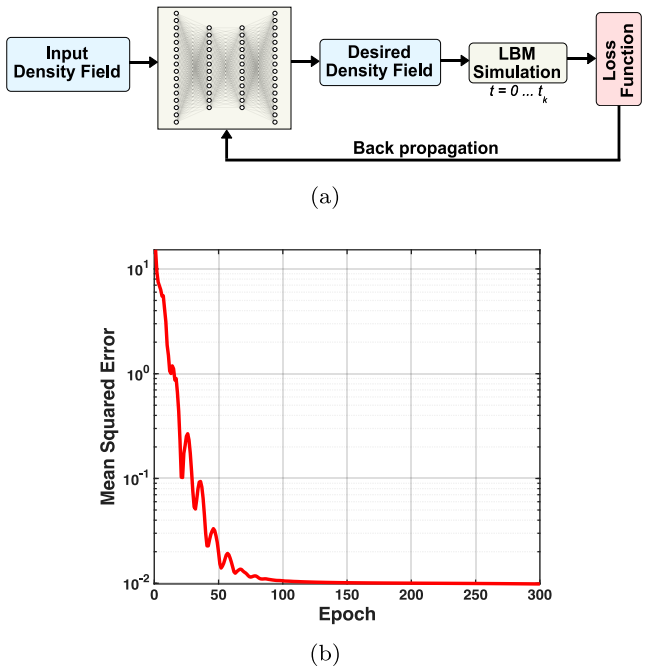


Figure 13. (a) Diagram of the workflow used to train the neural-network. (b) Mean-squared error (MSE) decay with training epochs.

For $t_k = 900$, the mean squared error during training is presented in Figure 13b, and the results obtained from solving the above optimization problem are shown in Figure 14.

This demonstration illustrates how automatic differentiation (Bradbury et al., 2018), when combined with the LBM framework, can be employed to address inverse problems in multiphase flows. More physically realistic results could potentially be achieved by augmenting the loss function in equation 25 with additional physically informed constraints such as Galilean and rotational invariance, mass and momentum conservation, among others (Corbetta et al., 2023). The framework can be extended to more complex geometries, such as porous media, where it may be applied for topology optimization to design structures with optimal flow and transport properties (Kong et al., 2025), or to establish correlations between pore-scale microstructure and flow behavior (Graczyk & Matyka, 2020), among other applications.

5. Performance Benchmarks

The performance of JAX-LaB was evaluated across various single-GPU and multi-GPU systems. Given the library's heavy dependence on JAX, the latest stable release available at the time of writing (version 0.6.0) was used to take advantage of recent performance enhancements and improvements. Simulations are performed using the D3Q19 lattice and the BGK collision model, with all I/O operations excluded. Unless specified otherwise, computations are carried out in single precision (f32/f32). The test case involves a droplet of

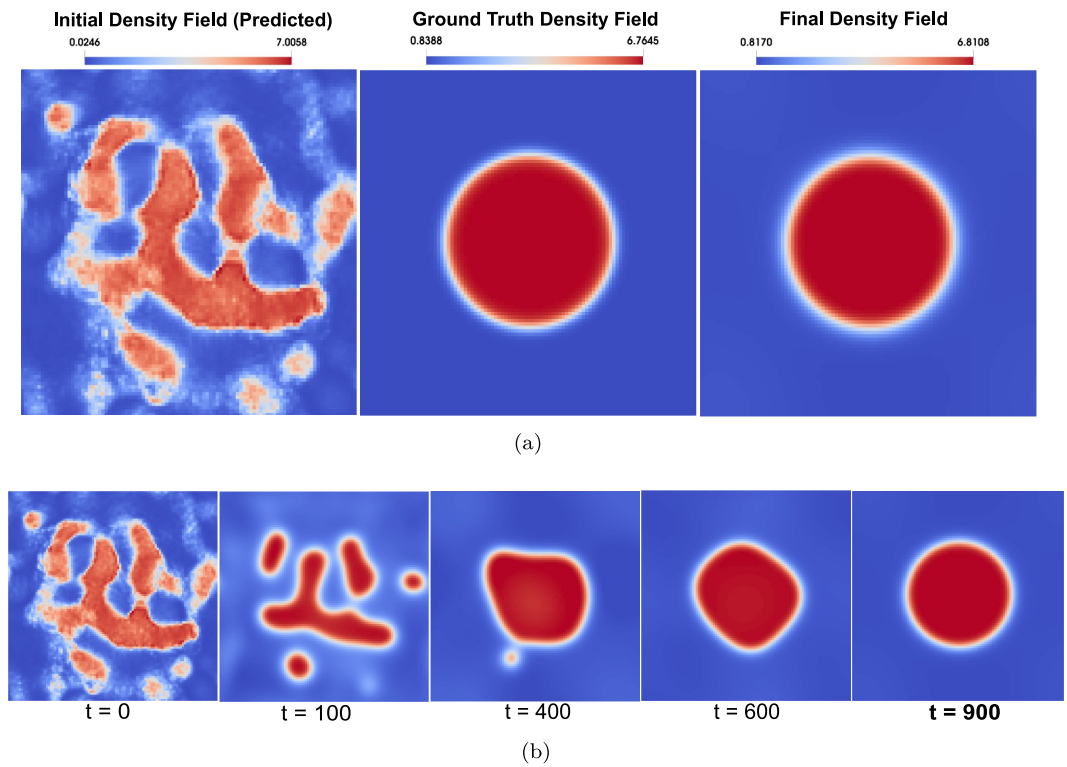


Figure 14. (a) Illustration of the initial density field predicted by the neural network, the target (ground truth) density field, and the resulting density field after 900 iterations. (b) Temporal evolution of the density field showing the initial distribution, and the formation of the droplet at $t = 900$.

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

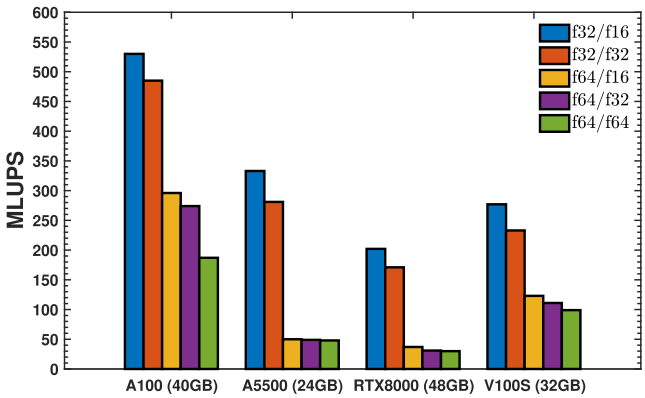


Figure 15. Single-GPU performance of JAX-LaB on different GPU architectures, across various mixed precision configurations (compute/write) for a droplet suspended in vapor.

radius 30 lattice units suspended in its vapor (Figure 3), using a pseudopotential defined as $\psi = \exp(-1/\rho)$, resulting in a liquid-vapor density ratio of approximately $\rho_l/\rho_v \approx 7.6$. Periodic boundary conditions are applied in all directions. This test case was selected for performance benchmarking because it involves a non-linear pseudopotential and can be easily scaled to match the GPU's computational capacity. The performance of the code is quantified using million lattice updates per second (MLUPS), which is defined as:

$$\text{MLUPS} = \frac{\text{No. of Components} \times nx \times ny \times nz}{\text{Compute Time(s)} \times 10^6} \tag{26}$$

During computation of MLUPS, the first iteration is not considered to account for the time taken for the just-in-time (JIT) compilation of the main-loop (Ataei & Salehipour, 2024).

5.1. Single GPU Performance

In this section, we evaluate the performance of JAX-LaB on single-GPU systems. Figure 15 presents MLUPS measurements across a range of GPUs, from desktop to server-grade, under different memory configurations and mixed-precision modes. The domain sizes are chosen to fully utilize the computational throughput of each device, though they do not reflect the maximum problem sizes each GPU is capable of handling. Across all tested hardware, mixed-precision (f32/f16) maximizes throughput but sacrifices numerical fidelity, making it unsuitable for simulations with high density ratios, where vapor-phase densities may be inaccurately represented. Single-precision (f32/f32) offers the best compromise between speed and accuracy, even under extreme density contrasts. Among the GPUs examined, the A100 (Ampere architecture) delivers the highest MLUPS in both single- and double-precision modes. As anticipated, double-precision workloads result in a significant performance drop across all GPUs evaluated in this study.

5.2. Multi-GPU Scaling

We evaluate the multi-GPU scaling performance of the JAX-LaB library on a DGX system equipped with A100 (40 GB) GPUs. The test setup mirrors the previously described configuration, but operates in single precision (f32/f32). Figure 16a illustrates the results of the weak scaling test, where the domain size increases proportionally with the number of GPUs along the x-direction, following the form $416n \times 416^2$. The results exhibit excellent scaling behavior, maintaining over 94% efficiency with 4 GPUs. Figure 16b presents the strong scaling performance for a fixed domain size of 320^3 as the GPU count increases. This test also shows high efficiency, achieving nearly 90% with 4 GPUs. These tests reveal the capability of JAX-LaB to model very large multiphase systems (4 GPU weak scaling tests involved more than 287 million points).

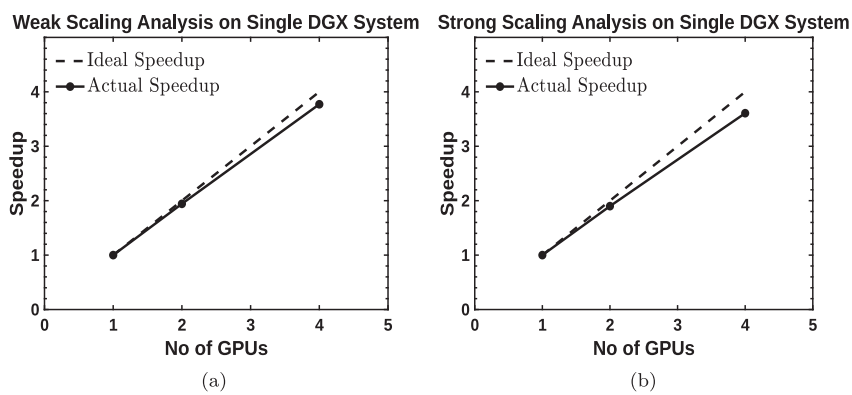


Figure 16. Scaling studies on A100 (40 GB) GPUs: (a) Weak scaling with domain of sizes $416n \times 416^2$ where n is the number of GPUs. (b) Strong scaling with cubic domain of size 320^3.

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

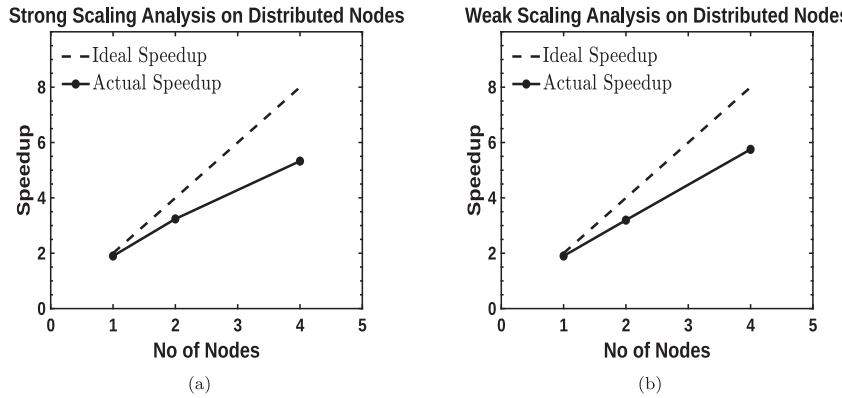


Figure 17. (a) Distributed scaling of JAX-LaB on a DGX A100 cluster (single precision): (a) Strong scaling with cubic domain of size 320^3. (b) Weak scaling with domain of sizes $412n \times 412^2$ where n is the number of nodes.

5.3. Distributed Scaling

In this section, we examine the distributed weak scaling performance of JAX-LaB on a cluster of DGX A100 (40 GB) GPU nodes, each equipped with two GPUs. The scaling test is conducted on up to four nodes, corresponding to a total of eight GPUs, using the same test configuration as in previous sections. All simulations are performed in single precision, with domain sizes scaled according to the number of available GPUs. To execute the code, individual JAX processes (initialized with jax.distributed.initialize) are launched on each node using the Message Passing Interface (MPI). MPI is the only configuration required for JAX-LaB to run on distributed systems. The results, presented in Figure 17, demonstrate scaling efficiency of 70% (weak scaling) and 66% (strong scaling) up to eight GPUs. The reduction in performance with an increasing number of nodes can be attributed to communication latency between nodes.

6. Conclusions

This paper introduces JAX-LaB, a JAX-based, multi-GPU LBM framework that extends XLB by incorporating support for multiphase and multicomponent systems, using the well-established Shan-Chen pseudopotential method. However, rather than relying on the original Shan-Chen formulation, JAX-LaB implements a revised version that integrates a suite of improvements proposed in the literature. These enhancements include: (a) direct incorporation of the equation of state, (b) an accurate force scheme capable of handling large density ratios while suppressing spurious currents, (c) a pressure-tensor modification technique that enables independent control of surface tension without altering the density ratio (a key limitation of the original model), and (d) an improved wetting boundary condition that supports a wide range of contact angles with minimal spurious velocities. These features are particularly valuable for simulating hydrological and geological flows, which often involve multiphase mixtures with diverse wettability and large liquid-vapor density ratios; for example, permeability, drainage, and imbibition simulations for sandstone and sphere pack porous media are showcased in this paper. Inheriting the modular and extensible design of XLB, JAX-LaB supports an arbitrary number of components by employing a list-based data structure (*pytrees*). The codebase has been validated against several analytical and numerical benchmarks across a variety of domains, demonstrating strong agreement in all cases. We demonstrated the differentiability of the code by utilizing the automatic differentiation capabilities of JAX to integrate machine learning with a physics-based code, employing this unified workflow to achieve fluid flow control through deep learning. These test cases are included as reproducible examples within the repository to facilitate user adoption and verification. Furthermore, the scalability of the codebase on massively parallel hardware such as GPUs has been rigorously benchmarked. These tests evaluate both single- and double-precision performance on various GPU architectures and characterize multi-GPU scalability of the codebase using weak and strong scaling tests for single and distributed GPU nodes.

JAX-LaB is actively maintained and distributed under the Apache License at https://github.com/piyush-ppradhan/JAX-LaB. Planned features include sparse array support to reduce memory overhead, implementation of thermal LBM for non-isothermal flows, and integration of turbulence modeling via LES. These developments aim

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

to extend JAX-LaB's applicability to a broad range of fluid flow modeling problems relevant to geoscience and engineering applications.

Conflict of Interest

The authors declare no conflicts of interest relevant to this study.

Data Availability Statement

JAX-LaB is released under the Apache License and is available at https://github.com/piyush-ppradhan/JAX-LaB. Since the library is under active development, the exact version of the code used to produce the results in this paper is made available separately at (Pradhan et al., 2025). This release includes the relevant JAX-LaB source code, example scripts, and visualization figures used in the various benchmarks and examples.

Acknowledgments
The single-GPU performance and multi-GPU scaling tests are performed on the Ginsburg cluster at Columbia University. The conceptualization and development of JAX-LaB by P.P., P.G., and S.K. was supported in part by the European Research Council (ERC) Synergy Grant "Understanding and modeling the Earth System with Machine Learning (USMILE)" under the Horizon 2020 research and innovation programme under Grant Agreement 855187 (P.G.) and the U. S. Department of Energy, Office of Science, Office of Basic Energy Sciences, Geosciences program under Award Number DE-SC0025348 (S.K.). P.G. acknowledges additional funding from the National Science Foundation Science and Technology Center, Learning the Earth with Artificial Intelligence and Physics, LEAP (Grant 2019625) and the Simons Foundation (Pivot Fellowship: From Galaxy Evolution to Climate Models: a Data-Driven Journey). S.K. acknowledges additional funding from the American Chemical Society Petroleum Research Fund Grant 66777-DNI9 and the Columbia University Qiu Zhong Wei Research Project fund.

References

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., et al. (2016). TensorFlow: A system for large-scale machine learning. *arXiv. 10.48550/arXiv.1605.08695*.

Aidun, C. K., & Clausen, J. R. (2010). Lattice-Boltzmann method for complex flows. *Annual Review of Fluid Mechanics*, *42*(1), 439–472. https://doi.org/10.1146/annurev-fluid-121108-145519

An, S., Erfani, H., Hellevang, H., & Niasar, V. (2021). Lattice-Boltzmann simulation of dissolution of carbonate rock during CO_2-saturated brine injection. *Chemical Engineering Journal*, *408*, 127235. https://doi.org/10.1016/j.cej.2020.127235

Ashirbekov, A., Kabdenova, B., Monaco, E., & Rojas-Solórzano, L. R. (2021). Equation of state's crossover enhancement of pseudopotential Lattice Boltzmann Modeling of CO_2 flow in homogeneous porous media. *Fluids*, *6*(12), 434. https://doi.org/10.3390/fluids6120434

Ataei, M., & Salehipour, H. (2023). XLB: Distributed multi-GPU Lattice Boltzmann simulation framework for differentiable scientific machine learning. *arXiv*. https://doi.org/10.48550/arXiv.2311.16080

Ataei, M., & Salehipour, H. (2024). XLB: A differentiable massively parallel lattice Boltzmann library in Python. *Computer Physics Communications*, *300*, 109187. https://doi.org/10.1016/j.cpc.2024.109187

Bartlett, C., Chen, H., Staroselsky, I., Wanderer, J., & Yakhot, V. (2013). Lattice Boltzmann two-equation model for turbulence simulations: High-Reynolds number flow past circular cylinder. *International Journal of Heat and Fluid Flow*, *42*, 1–9. https://doi.org/10.1016/j.ijheatfluidflow.2013.01.018

Bauer, M., Eibl, S., Godenschwager, C., Kohl, N., Kuron, M., Rettinger, C., et al. (2021). waLBerla: A block-structured high-performance framework for multiphysics simulations. *Computers & Mathematics with Applications*, *81*, 478–501. https://doi.org/10.1016/j.camwa.2020.01.007

Bedrunka, M. C., Horstmann, T., Picard, B., Reith, D., & Foysi, H. (2024). Machine learning enhanced collision operator for the lattice Boltzmann method based on invariant networks. *arXiv*. https://doi.org/10.48550/arXiv.2412.08229

Bedrunka, M. C., Wilde, D., Kliemank, M., Reith, D., Foysi, H., & Krämer, A. (2021). Lettuce: PyTorch-Based lattice Boltzmann framework. In H. Jagode, H. Anzt, H. Ltaief, & P. Luszczek (Eds.), *High performance computing* (pp. 40–55). Springer International Publishing. https://doi.org/10.1007/978-3-030-90539-2_3

Bezgin, D. A., Buhendwa, A. B., & Adams, N. A. (2023). JAX-Fluids: A fully-differentiable high-order computational fluid dynamics solver for compressible two-phase flows. *Computer Physics Communications*, *282*, 108527. https://doi.org/10.1016/j.cpc.2022.108527

Bezgin, D. A., Buhendwa, A. B., & Adams, N. A. (2025). JAX-fluids 2.0: Towards HPC for differentiable CFD of compressible two-phase flows. *Computer Physics Communications*, *308*, 109433. https://doi.org/10.1016/j.cpc.2024.109433

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., et al. (2018). JAX: Composable transformations of Python+NumPy programs. Retrieved from http://github.com/jax-ml/jax

Calzavarini, E. (2019). Eulerian–Lagrangian fluid dynamics platform: The ch4-project. *Software Impacts*, *1*, 100002. https://doi.org/10.1016/j.simpa.2019.100002

Chaaban, M., Heider, Y., Sun, W., & Markert, B. (2024). A machine-learning supported multi-scale LBM-TPM model of unsaturated, anisotropic, and deformable porous materials. *International Journal for Numerical and Analytical Methods in Geomechanics*, *48*(4), 889–910. https://doi.org/10.1002/nag.3668

Chen, Y., Valocchi, A. J., Kang, Q., & Viswanathan, H. S. (2019). Inertial effects during the process of supercritical CO_2 displacing brine in a sandstone: Lattice Boltzmann simulations based on the continuum-surface-force and geometrical wetting models. *Water Resources Research*, *55*(12), 11144–11165. https://doi.org/10.1029/2019WR025746

Coon, E. T., Porter, M. L., & Kang, Q. (2014). Taxila LBM: A parallel, modular lattice Boltzmann framework for simulating pore-scale flow in porous media. *Computational Geosciences*, *18*(1), 17–27. https://doi.org/10.1007/s10596-013-9379-6

Corbetta, A., Gabbana, A., Gyrya, V., Livescu, D., Prins, J., & Toschi, F. (2023). Toward learning Lattice Boltzmann collision operators. *The European Physical Journal E*, *46*(3), 10. https://doi.org/10.1140/epje/s10189-023-00267-w

Coveney, P. V., Succi, S., d'Humières, D., Ginzburg, I., Krafczyk, M., Lallemand, P., & Luo, L.-S. (2002). Multiple–relaxation–time lattice Boltzmann models in three dimensions. *Philosophical Transactions of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, *360*(1792), 437–451. https://doi.org/10.1098/rsta.2001.0955

Ding, H., & Spelt, P. D. M. (2007). Wetting condition in diffuse interface simulations of contact line motion. *Physical Review E*, *75*(4), 046708. https://doi.org/10.1103/PhysRevE.75.046708

Fei, L., & Luo, K. H. (2017). Consistent forcing scheme in the cascaded lattice Boltzmann method. *Physical Review E*, *96*(5), 053307. https://doi.org/10.1103/PhysRevE.96.053307

Fei, L., Qin, F., Zhao, J., Derome, D., & Carmeliet, J. (2022). Pore-scale study on convective drying of porous media. *Langmuir*, *38*(19), 6023–6035. https://doi.org/10.1021/acs.langmuir.2c00267

Feng, Y., Boivin, P., Jacob, J., & Sagaut, P. (2019). Hybrid recursive regularized lattice Boltzmann simulation of humid air with application to meteorological flows. *Physical Review E*, *100*(2), 023304. https://doi.org/10.1103/PhysRevE.100.023304

Filippova, O., Succi, S., Mazzocco, F., Arrighetti, C., Bella, G., & Hänel, D. (2001). Multiscale Lattice Boltzmann schemes with turbulence modeling. *Journal of Computational Physics*, *170*(2), 812–829. https://doi.org/10.1006/jcph.2001.6764

Galindo-Torres, S. A., Scheuermann, A., & Li, L. (2016). Boundary effects on the soil water characteristic curves obtained from lattice Boltzmann simulations. *Computers and Geotechnics*, *71*, 136–146. https://doi.org/10.1016/j.compgeo.2015.09.008

Gao, J., Xing, H., Tian, Z., Pearce, J. K., Sedek, M., Golding, S. D., & Rudolph, V. (2017). Reactive transport in porous media for CO_2 sequestration: Pore scale modeling using the lattice Boltzmann method. *Computers & Geosciences*, *98*, 9–20. https://doi.org/10.1016/j.cageo.2016.09.008

Geier, M., Greiner, A., & Korvink, J. G. (2006). Cascaded digital lattice Boltzmann automata for high Reynolds number flow. *Physical Review E*, *73*(6), 066705. https://doi.org/10.1103/PhysRevE.73.066705

Gong, S., & Cheng, P. (2012). Numerical investigation of droplet motion and coalescence by an improved lattice Boltzmann model for phase transitions and multiphase flows. *Computers & Fluids*, *53*, 93–104. https://doi.org/10.1016/j.compfluid.2011.09.013

Graczyk, K. M., & Matyka, M. (2020). Predicting porosity, permeability, and tortuosity of porous media from images by deep learning. *Scientific Reports*, *10*(1), 21488. https://doi.org/10.1038/s41598-020-78415-x

Gunstensen, A. K., Rothman, D. H., Zaleski, S., & Zanetti, G. (1991). Lattice Boltzmann model of immiscible fluids. *Physical Review A*, *43*(8), 4320–4327. https://doi.org/10.1103/PhysRevA.43.4320

Halpern, D., & Gaver, D. P. (1994). Boundary element analysis of the time-dependent motion of a semi-infinite bubble in a channel. *Journal of Computational Physics*, *115*(2), 366–375. https://doi.org/10.1006/jcph.1994.1202

Hamraoui, A., Thuresson, K., Nylander, T., & Yaminsky, V. (2000). Can a dynamic contact angle be understood in terms of a friction coefficient? *Journal of Colloid and Interface Science*, *226*(2), 199–204. https://doi.org/10.1006/jcis.2000.6830

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., et al. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hennigh, O. (2023). Phantomgaze: A GPU-accelerated rendering engine for scientific visualization. Retrieved from https://github.com/loliverhennigh/PhantomGaze

Heshmati, M., & Piri, M. (2014). Experimental investigation of dynamic contact angle and capillary rise in tubes with circular and noncircular cross sections. *Langmuir*, *30*(47), 14151–14162. https://doi.org/10.1021/la501724y

Horstmann, J. T., Bedrunka, M. C., & Foysi, H. (2024). Lattice Boltzmann method with artificial bulk viscosity using a neural collision operator. *Computers & Fluids*, *272*, 106191. https://doi.org/10.1016/j.compfluid.2024.106191

Hu, A., Li, L., & Uddin, R. (2014). Surface tension adjustment in a pseudo-potential lattice Boltzmann model. *arXiv*. https://doi.org/10.48550/arXiv.1412.7228

Hu, A., Li, L., Uddin, R., & Liu, D. (2016). Contact angle adjustment in equation-of-state-based pseudopotential model. *Physical Review E*, *93*(5), 053307. https://doi.org/10.1103/PhysRevE.93.053307

Januszewski, M., & Kostur, M. (2014). Sailfish: A flexible multi-GPU implementation of the lattice Boltzmann method. *Computer Physics Communications*, *185*(9), 2350–2368. https://doi.org/10.1016/j.cpc.2014.04.018

Kang, Q., Zhang, D., & Chen, S. (2004). Immiscible displacement in a channel: Simulations of fingering in two dimensions. *Advances in Water Resources*, *27*(1), 13–22. https://doi.org/10.1016/j.advwatres.2003.10.002

Karlin, I. V., Bösch, F., & Chikatamarla, S. S. (2014). Gibbs' principle for the lattice-kinetic theory of fluid dynamics. *Physical Review E*, *90*(3), 031302. https://doi.org/10.1103/PhysRevE.90.031302

Kelly, S., El-Sobky, H., Torres-Verdín, C., & Balhoff, M. T. (2016). Assessing the utility of FIB-SEM images for shale digital rock physics. *Advances in Water Resources*, *95*, 302–316. https://doi.org/10.1016/j.advwatres.2015.06.010

Kidger, P., & Garcia, C. (2021). Equinox: Neural networks in JAX via callable PyTrees and filtered transformations. *arXiv*. https://doi.org/10.48550/arXiv.2111.00254

Kingma, D. P., & Ba, J. (2017). Adam: A method for stochastic optimization. *arXiv. 10.48550/arXiv.1412.6980*.

Kochkov, D., Smith, J. A., Alieva, A., Wang, Q., Brenner, M. P., & Hoyer, S. (2021). Machine learning–accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, *118*(21), e2101784118. https://doi.org/10.1073/pnas.2101784118

Kong, X., Zhang, H., Du, Y., Wang, X., & Xiao, G. (2025). Adjoint lattice Boltzmann method-based multiscale topology optimization of thermo-fluid systems using anisotropic porous media. *Physics of Fluids*, *37*(8), 087208. https://doi.org/10.1063/5.0279717

Krause, M. J., Kummerländer, A., Avis, S. J., Kusumaatmaja, H., Dapelo, D., Klemens, F., et al. (2021). OpenLB—Open source lattice Boltzmann code. *Computers & Mathematics with Applications*, *81*, 258–288. https://doi.org/10.1016/j.camwa.2020.04.033

Kupershtokh, A., Medvedev, D., & Karpov, D. (2009). On equations of state in a lattice Boltzmann method. *Computers & Mathematics with Applications*, *58*(5), 965–974. https://doi.org/10.1016/j.camwa.2009.02.024

Latt, J., Malaspinas, O., Kontaxakis, D., Parmigiani, A., Lagrava, D., Brogi, F., et al. (2021). Palabos: Parallel lattice Boltzmann solver. *Computers & Mathematics with Applications*, *81*, 334–350. https://doi.org/10.1016/j.camwa.2020.03.022

Lehmann, M. (2019). High performance free surface LBM on GPUs. (Doctoral dissertation). https://doi.org/10.15495/EPub_UBT_00005400

Lenormand, R., Touboul, E., & Zarcone, C. (1988). Numerical models and experiments on immiscible displacements in porous media. *Journal of Fluid Mechanics*, *189*, 165–187. https://doi.org/10.1017/S0022112088000953

Li, K., Zhong, C., Zhuo, C., & Cao, J. (2012). Non-body-fitted Cartesian-mesh simulation of highly turbulent flows using multi-relaxation-time lattice Boltzmann method. *Computers & Mathematics with Applications*, *63*(10), 1481–1496. https://doi.org/10.1016/j.camwa.2012.03.080

Li, Q., & Luo, K. H. (2013). Achieving tunable surface tension in the pseudopotential lattice Boltzmann modeling of multiphase flows. *Physical Review E*, *88*(5), 053307. https://doi.org/10.1103/PhysRevE.88.053307

Li, Q., Yu, Y., & Luo, K. H. (2019). Implementation of contact angles in pseudopotential lattice Boltzmann simulations with curved boundaries. *Physical Review E*, *100*(5), 053313. https://doi.org/10.1103/PhysRevE.100.053313

Linhao, Z., Shide, F., & Shouting, G. (2005). Wind-driven ocean circulation in shallow water lattice Boltzmann model. *Advances in Atmospheric Sciences*, *22*(3), 349–358. https://doi.org/10.1007/BF02918749

Liu, S., Zhang, C., & Ghahfarokhi, R. B. (2021). A review of lattice-Boltzmann models coupled with geochemical modeling applied for simulation of advanced waterflooding and enhanced oil recovery processes. *Energy & Fuels*, *35*(17), 13535–13549. https://doi.org/10.1021/acs.energyfuels.1c01347

Lou, Q., Guo, Z., & Shi, B. (2013). Evaluation of outflow boundary conditions for two-phase lattice Boltzmann equation. *Physical Review E*, *87*(6), 063301. https://doi.org/10.1103/PhysRevE.87.063301

Luebke, D. (2008). CUDA: Scalable parallel programming for high-performance scientific computing. In *2008 5th IEEE International Symposium on Biomedical Imaging: From Nano to Macro* (pp. 836–838). https://doi.org/10.1109/ISBI.2008.4541126

Luo, K. H., Fei, L., & Wang, G. (2021). A unified lattice Boltzmann model and application to multiphase flows. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, *379*(2208), 20200397. https://doi.org/10.1098/rsta.2020.0397

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

McCracken, M. E., & Abraham, J. (2005). Multiple-relaxation-time lattice-Boltzmann model for multiphase flow. *Physical Review E*, *71*(3), 036701. https://doi.org/10.1103/PhysRevE.71.036701

Mittal, R., & Iaccarino, G. (2005). Immersed boundary methods. *Annual Review of Fluid Mechanics*, *37*(1), 239–261. https://doi.org/10.1146/annurev.fluid.37.061903.175743

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *arXiv. 10.48550/arXiv.1912.01703*.

Pellerin, N., Leclaire, S., & Reggio, M. (2015). An implementation of the Spalart–Allmaras turbulence model in a multi-domain lattice Boltzmann method for solving turbulent airfoil flows. *Computers & Mathematics with Applications*, *70*(12), 3001–3018. https://doi.org/10.1016/j.camwa.2015.10.006

Petersen, K. J., & Brinkerhoff, J. R. (2021). On the lattice Boltzmann method and its application to turbulent, multiphase flows of various fluids including cryogens: A review. *Physics of Fluids*, *33*(4), 041302. https://doi.org/10.1063/5.0046938

Pitsch, H. (2006). Large-eddy simulation of turbulent combustion. *Annual Review of Fluid Mechanics*, *38*(1), 453–482. https://doi.org/10.1146/annurev.fluid.38.050304.092133

Porter, M. L., Coon, E. T., Kang, Q., Moulton, J. D., & Carey, J. W. (2012). Multicomponent interparticle-potential lattice Boltzmann model for fluids with large viscosity ratios. *Physical Review E*, *86*(3), 036701. https://doi.org/10.1103/PhysRevE.86.036701

Porter, M. L., Schaap, M. G., & Wildenschild, D. (2009). Lattice-Boltzmann simulations of the capillary pressure–saturation–interfacial area relationship for porous media. *Advances in Water Resources*, *32*(11), 1632–1640. https://doi.org/10.1016/j.advwatres.2009.08.009

Pradhan, P., Gentine, P., & Kelly, S. (2025). JAX-LaB: A Python-based, accelerated, differentiable massively parallel lattice Boltzmann library for modeling multiphase and multiphysics flows & physics-based machine learning. https://doi.org/10.5281/zenodo.17613708

Premnath, K. N., Pattison, M. J., & Banerjee, S. (2009). Dynamic subgrid scale modeling of turbulent flows using lattice-Boltzmann method. *Physica A: Statistical Mechanics and its Applications*, *388*(13), 2640–2658. https://doi.org/10.1016/j.physa.2009.02.041

Qin, Z., Zhu, J., Chen, W., Li, C., & Wen, B. (2022). An effective pseudo-potential lattice Boltzmann model with extremely large density ratio and adjustable surface tension. *Physics of Fluids*, *34*(11), 113328. https://doi.org/10.1063/5.0123727

Santos, J. E., Chang, B., Kang, Q., Viswanathan, H., Lubbers, N., Gigliotti, A., & Prodanovic, M. (2025). 3D dataset of simulations. https://doi.org/10.17612/93pd-y471

Santos, J. E., Gigliotti, A., Bihani, A., Landry, C., Hesse, M. A., Pyrcz, M. J., & Prodanović, M. (2022). MPLBM-UT: Multiphase LBM library for permeable media analysis. *SoftwareX*, *18*, 101097. https://doi.org/10.1016/j.softx.2022.101097

Santos, J. E., Pyrcz, M. J., & Prodanović, M. (2022). 3D dataset of binary images: A collection of synthetically created digital rock images of complex media. *Data in Brief*, *40*, 107797. https://doi.org/10.1016/j.dib.2022.107797

Sbragaglia, M., Benzi, R., Biferale, L., Succi, S., Sugiyama, K., & Toschi, F. (2007). Generalized lattice Boltzmann method with multirange pseudopotential. *Physical Review E*, *75*(2), 026702. https://doi.org/10.1103/PhysRevE.75.026702

Schindelin, J., Arganda-Carreras, I., Frise, E., Kaynig, V., Longair, M., Pietzsch, T., et al. (2012). Fiji: An open-source platform for biological-image analysis. *Nature Methods*, *9*(7), 676–682. https://doi.org/10.1038/nmeth.2019

Shan, X. (2008). Pressure tensor calculation in a class of nonideal gas lattice Boltzmann models. *Physical Review E*, *77*(6), 066702. https://doi.org/10.1103/PhysRevE.77.066702

Shan, X., & Chen, H. (1993). Lattice Boltzmann model for simulating flows with multiple phases and components. *Physical Review E*, *47*(3), 1815–1819. https://doi.org/10.1103/PhysRevE.47.1815

Siebold, A., Nardin, M., Schultz, J., Walliser, A., & Oppliger, M. (2000). Effect of dynamic contact angle on capillary rise phenomena. *Colloids and Surfaces A: Physicochemical and Engineering Aspects*, *161*(1), 81–87. https://doi.org/10.1016/S0927-7757(99)00327-1

Song, W., Zhang, Y., Li, B., & Fan, X. (2016). A lattice Boltzmann model for heat and mass transfer phenomena with phase transformations in unsaturated soil during freezing process. *International Journal of Heat and Mass Transfer*, *94*, 29–38. https://doi.org/10.1016/j.ijheatmasstransfer.2015.11.008

Stone, J. E., Gohara, D., & Shi, G. (2010). OpenCL: A parallel programming standard for heterogeneous computing systems. *Computing in Science & Engineering*, *12*(3), 66–73. https://doi.org/10.1109/MCSE.2010.69

Sukop, M. C., & Thorne, D. T. (2006). *Lattice Boltzmann modeling: An introduction for geoscientists and engineers*. Springer. https://doi.org/10.1007/978-3-540-27982-2

Swift, M. R., Orlandini, E., Osborn, W. R., & Yeomans, J. M. (1996). Lattice Boltzmann simulations of liquid-gas and binary fluid systems. *Physical Review E*, *54*(5), 5041–5052. https://doi.org/10.1103/PhysRevE.54.5041

Tang, Y., Qiao, Z., Cao, Y., Si, F., & Zhang, C. (2024). Multi-component multiphase lattice Boltzmann modeling of water purging during supercritical carbon dioxide extraction from geothermal reservoir pores. *Renewable Energy*, *220*, 119660. https://doi.org/10.1016/j.renene.2023.119660

Tian, Z., Fei, L., Wang, C., Guo, K., Tian, W., Qiu, S., et al. (2025). Lattice Boltzmann modeling of evaporation of porous media considering conjugate heat transfer. *Physics of Fluids*, *37*(3), 033351. https://doi.org/10.1063/5.0261217

Vinuesa, R., & Brunton, S. L. (2022). Enhancing computational fluid dynamics with machine learning. *Nature Computational Science*, *2*(6), 358–366. https://doi.org/10.1038/s43588-022-00264-7

Wang, H., Xu, S., Qu, Z., Wang, H., Xu, S., & Qu, Z. (2024). *Machine learning assisting intelligent control of evaporation performance in porous media*. IntechOpen. https://doi.org/10.5772/intechopen.1007473

Wang, Y. D., Chung, T., Armstrong, R. T., & Mostaghimi, P. (2021). ML-LBM: Predicting and accelerating steady state flow simulation in porous media with convolutional neural networks. *Transport in Porous Media*, *138*(1), 49–75. https://doi.org/10.1007/s11242-021-01590-6

Washburn, E. W. (1921). The dynamics of capillary flow. *Physical Review*, *17*(3), 273–283. https://doi.org/10.1103/PhysRev.17.273

Wolf, F. G., Dos Santos, L. O., & Philippi, P. C. (2010). Capillary rise between parallel plates under dynamic conditions. *Journal of Colloid and Interface Science*, *344*(1), 171–179. https://doi.org/10.1016/j.jcis.2009.12.023

Wörner, M. (2012). Numerical modeling of multiphase flows in microfluidics and micro process engineering: A review of methods and applications. *Microfluidics and Nanofluidics*, *12*(6), 841–886. https://doi.org/10.1007/s10404-012-0940-8

Xie, J.-F., He, S., Zu, Y. Q., Lamy-Chappuis, B., & Yardley, B. W. D. (2017). Relative permeabilities of supercritical CO_2 and brine in carbon sequestration by a two-phase lattice Boltzmann method. *Heat and Mass Transfer*, *53*(8), 2637–2649. https://doi.org/10.1007/s00231-017-2007-6

Xu, R., Prodanović, M., & Landry, C. (2020). Pore-scale study of water adsorption and subsequent methane transport in clay in the presence of wettability heterogeneity. *Water Resources Research*, *56*(10), e2020WR027568. https://doi.org/10.1029/2020WR027568

Yang, J., Xu, Y., & Yang, L. (2022). Taichi-LBM3D: A single-phase and multiphase lattice Boltzmann solver on cross-platform multicore CPU/GPUs. *Fluids*, *7*(8), 270. https://doi.org/10.3390/fluids7080270

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Yang, X., Chang, C., Zheng, M., Wang, X., Chen, Y., Xie, W., et al. (2025). Characterizing dynamic contact angle during gas–liquid imbibition in microchannels by lattice Boltzmann method modeling. *ACS Omega*, *10*(3), 3116–3127. https://doi.org/10.1021/acsomega.4c10365

Yang, Y., Horne, R. N., Cai, J., & Yao, J. (2023). Recent advances on fluid flow in porous media using digital core analysis technology. *Advances in Geo-Energy Research*, *9*(2), 71–75. https://doi.org/10.46690/ager.2023.08.01

Yoshino, M., Sasaki, K., Saito, S., & Suzuki, K. (2022). Lattice Boltzmann simulation of behaviors of binary cloud droplets approaching each other. *Multiphase Science and Technology*, *34*(3), 1–15. https://doi.org/10.1615/MultScienTechn.2022043604

Yuan, P., & Schaefer, L. (2006). Equations of state in a lattice Boltzmann model. *Physics of Fluids*, *18*(4), 042101. https://doi.org/10.1063/1.2187070

Zhang, J. (2011). Lattice Boltzmann method for microfluidics: Models and applications. *Microfluidics and Nanofluidics*, *10*(1), 1–28. https://doi.org/10.1007/s10404-010-0624-1

Zhang, R., & Chen, H. (2003). Lattice Boltzmann method for simulations of liquid-vapor thermal flows. *Physical Review E*, *67*(6), 066711. https://doi.org/10.1103/PhysRevE.67.066711

Zhang, T., Sun, S., & Song, H. (2019). Flow mechanism and simulation approaches for shale gas reservoirs: A review. *Transport in Porous Media*, *126*(3), 655–681. https://doi.org/10.1007/s11242-018-1148-5

Zhou, Y., Chen, L., Gong, Y., & Wang, S. (2021). Pore-scale simulations of particles migration and deposition in porous media using LBM-DEM coupling method. *Processes*, *9*(3), 465. https://doi.org/10.3390/pr9030465

Zhu, J., Dai, Z., Wang, Z., Chu, S., & Wang, M. (2024). Investigation on enhanced density ratio recovery and numerical stability in real physical field under multi-component multiphase LBM. *International Communications in Heat and Mass Transfer*, *156*, 107673. https://doi.org/10.1016/j.icheatmasstransfer.2024.107673

19422466, 2026, 2, Downloaded from https://agupubs.onlinelibrary.wiley.com/doi/10.1029/2025MS005313 by Capes, Wiley Online Library on [14/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License